Data Analytics (BCS-052)

Unit 1: Data Analytics Lifecycle & Big Data

Q1. Short Definitions (2 Marks Each)

(a) Differentiate between structured, semi-structured and unstructured data.

Basis	Structured	Semi-structured	Unstructured
Format	Fixed schema: rows and columns	Tags or keys, flexible schema	No predefined format
Storage	RDBMS (MySQL, Oracle)	XML, JSON files, NoSQL (MongoDB)	Files, HDFS, data lakes
Querying	Easy (SQL)	Moderate (XPath, JSON queries)	Hard (needs NLP, image processing)
Share of all data	About 10%	About 10%	**About 80%**
Examples	Bank transactions, student records	Emails (header + body), XML, JSON, CSV logs	Images, videos, audio, social-media posts, PDFs

(b) Define the core characteristics (the "V"s) of Big Data.

- **Volume:** the **huge size** of data, in terabytes to zettabytes. Example: Facebook stores petabytes of photos.
- **Velocity:** the **speed** at which data is generated and must be processed. Example: stock ticks, sensor streams.
- **Variety: different types and formats**: structured, semi-structured and unstructured.
- **Veracity:** the **uncertainty, noise and trustworthiness** of the data.
- **Value:** the **useful business insight** that can be extracted.
- *(Also mentioned sometimes: **Variability**, meaning inconsistent data flow, and **Visualization**.)*

(c) Differentiate between data analysis and reporting.

Data Analysis	Reporting
Explains **why** something happened and **what will happen**	Shows **what** happened
Turns information into **insights and recommendations**	Turns raw data into **organised summaries**
Interactive, exploratory, ad-hoc	Fixed format, scheduled (daily or monthly)
Needs statistics, ML and domain expertise	Needs standard BI tools and dashboards
Example: finding the reason why sales dropped in the North region	Example: monthly sales report by region

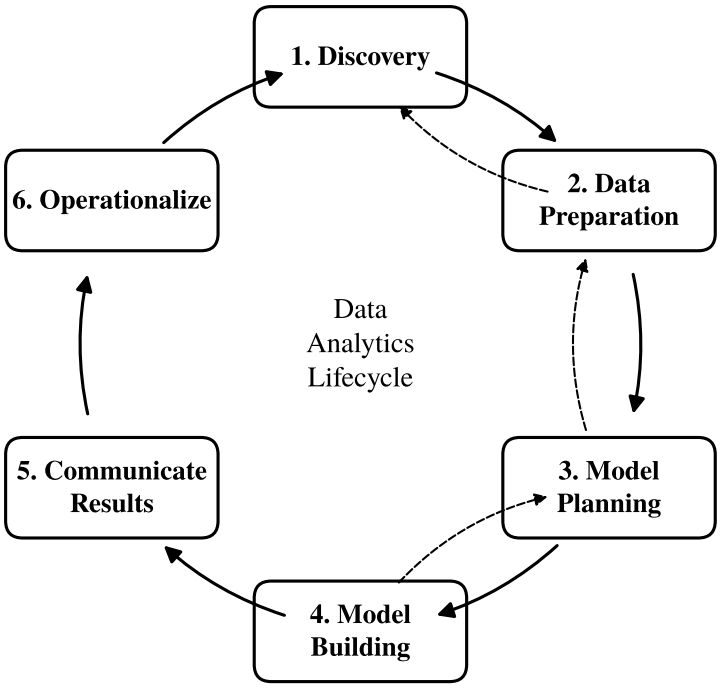
---

Q2. Explain the 6 phases of the Data Analytics Lifecycle in detail with a diagram. (7 Marks)

1. Introduction / Definition

The **Data Analytics Lifecycle** (the EMC / Dell model) is a **six-phase, iterative process** that guides a data science project from understanding the business problem to deploying the model in production. The phases are **Discovery, Data Preparation, Model Planning, Model Building, Communicating Results and Operationalization**. A team can move **back to an earlier phase** whenever new information appears.

2. Block Diagram



Data Analytics Lifecycle: solid arrows show the normal flow, dashed arrows show going back to an earlier phase

3. Core Explanation / Working

Phase 1: Discovery

- Understand the **business domain and the problem**. Frame it as an **analytics problem**.
- Identify the **stakeholders**, the resources (people, tools, data, time) and the success criteria.
- Form **initial hypotheses (IH)** to test. Example: “Customers who call support more than 3 times are likely to churn”.
- **Exit question:** do we have enough information to draft an analytic plan?

Phase 2: Data Preparation

- Set up an **analytic sandbox** (a separate workspace, so the production systems are not disturbed).
- Perform **ETLT** (Extract, Transform, Load, Transform): collect, clean, integrate and transform the data.
- Handle **missing values, outliers and duplicates**. Do **data conditioning** and visualization to understand the data.
- Tools: **Hadoop, Alpine Miner, OpenRefine, Data Wrangler**.
- This phase usually takes **50 to 80% of the project time**.

Phase 3: Model Planning

- Choose the **methods, techniques and workflow**: classification, clustering, regression and so on.

- Do **exploratory data analysis (EDA)** to find relationships between variables. Do **feature selection**.
- Tools: **R, SQL Analysis Services, SAS/ACCESS**.

Phase 4: Model Building

- Split the data into **training, testing and validation** sets.
- **Build and run the models** (for example decision tree, random forest, neural network). Tune the parameters.
- Check whether the model is **robust enough** and whether more computing power is needed.
- Tools: **R, Python (scikit-learn), SAS Enterprise Miner, MATLAB, Spark MLlib**.

Phase 5: Communicate Results

- Compare the outcomes with the **success criteria** from Phase 1. Decide whether the hypotheses are **accepted or rejected**.
- Present the **key findings, business value and limitations** to stakeholders, using **visualizations** and a clear story.

Phase 6: Operationalize

- Deliver the **final reports, code and technical documents**.
- Run a **pilot project** in a production-like environment, then **deploy** the model.
- **Monitor** its performance and **retrain** it when the data changes (model drift).

4. Key Characteristics - Phase Summary

Phase	Key Activity	Main Output	Key Roles
1. Discovery	Frame the problem, form hypotheses	Problem statement, initial hypotheses	Business user, project sponsor
2. Data Preparation	Sandbox, ETLT, cleaning	Clean, conditioned dataset	Data engineer, DBA
3. Model Planning	Select techniques and variables (EDA)	Model plan	Data scientist
4. Model Building	Train, test, tune	Working model	Data scientist
5. Communicate Results	Evaluate and present	Findings and recommendations	Data scientist, business analyst
6. Operationalize	Pilot, deploy, monitor	Production system, documentation	Data engineer, BI analyst

5. Advantages & Disadvantages

Advantages

- Gives a **structured, repeatable** approach that reduces project failure.
- Its **iterative** nature lets the team go back and fix gaps early.
- Defines clear **roles and deliverables** for each phase.

Disadvantages

- Data preparation is **time-consuming**, and the project may loop for a long time.
- It needs **skilled people** in many roles, and its **cost** is high for small projects.

6. Conclusion / Application

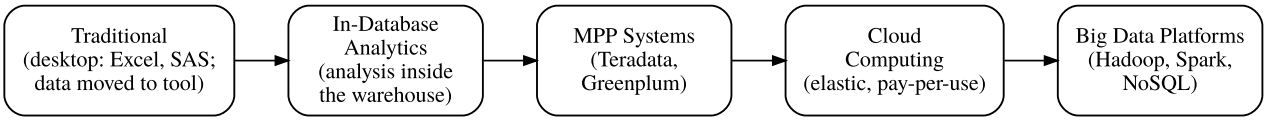
The lifecycle is followed in projects like **customer-churn prediction in telecom, fraud detection in banking and demand forecasting in retail**.

Q3. Explain the evolution of analytic scalability and the need for Big Data platforms. (7 Marks)

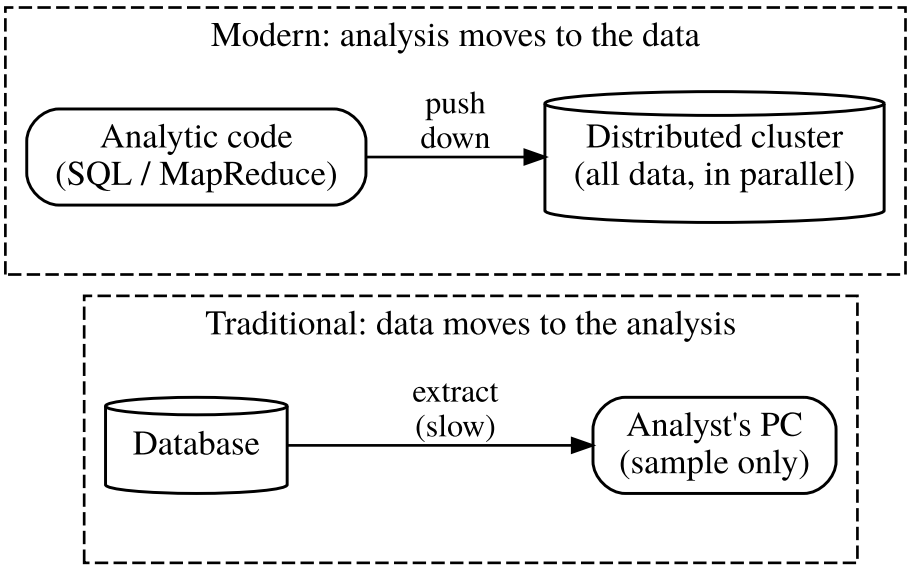
1. Introduction / Definition

Analytic scalability is the ability of an analytics system to **handle growing data volumes and workloads** by adding resources, while keeping performance acceptable. Over time, analytics has moved from **single desktop tools** to **in-database analytics, MPP systems, cloud computing and distributed Big Data platforms** such as Hadoop and Spark.

2. Block Diagram - Evolution



Evolution of analytic scalability



Old approach vs. modern approach: move the analysis to the data

3. Core Explanation / Working

(a) Traditional analytics (the 1990s)

- Data was **extracted** from databases to an analyst's desktop or server (SAS, SPSS, Excel).
- **Problems:** moving data is slow, only **samples** can be analysed, there is a single machine bottleneck, and security risks arise from copies.

(b) In-database analytics

- Analytic functions (scoring, statistics) run **inside the database or data warehouse**, using SQL and user-defined functions.
- This **removes data movement** and uses the database's processing power.

(c) Massively Parallel Processing (MPP)

- Data is **partitioned across many nodes**. Each node processes its part **in parallel** (a shared-nothing architecture).
- Examples: **Teradata, Greenplum, Netezza**. Performance scales almost **linearly** as nodes are added.

(d) Cloud computing

- Gives **on-demand, elastic** resources (IaaS, PaaS, SaaS), so there is no up-front hardware cost.
- Examples: AWS EMR, Google BigQuery, Azure Synapse.

(e) Big Data platforms (Hadoop and Spark)

- **Distributed storage (HDFS)** and **distributed processing (MapReduce, Spark)** on **commodity hardware**.
- They handle **all types of data** (structured and unstructured) at **petabyte scale**, with fault tolerance.
- **Data locality:** computation is sent to the node where the data lives.

(f) Need for Big Data platforms

1. **Volume:** traditional RDBMSs cannot store or process petabytes economically.
2. **Variety:** RDBMSs need a fixed schema, while most data today is unstructured.
3. **Velocity:** real-time streams need parallel, low-latency processing.
4. **Cost: horizontal scaling** with cheap commodity servers is cheaper than vertical scaling with bigger servers.
5. **Fault tolerance:** data is **replicated**, so a node failure does not lose data.
6. **Advanced analytics:** ML on the **full dataset** (not samples) gives better models.

4. Key Characteristics - Traditional vs. Big Data Platform

Basis	Traditional Analytics	Big Data Platform
Scaling	**Vertical** (bigger server)	**Horizontal** (more nodes)
Data types	Structured only	Structured, semi-structured and unstructured
Schema	Schema-on-write	**Schema-on-read**
Data size	GB to a few TB	TB to PB
Hardware	Expensive, specialised	Cheap commodity machines
Processing	Centralised	Distributed, parallel
Fault tolerance	Limited	Built in (replication)

5. Advantages & Disadvantages

Advantages

- Analyses the **entire dataset** quickly, with near-linear scalability.
- **Cost-effective** storage and processing on commodity hardware.
- Supports **real-time and advanced** analytics.

Disadvantages

- **Complex** to set up and manage, and needs specialised skills.
- **Security, privacy and data governance** are harder in distributed systems.

6. Conclusion / Application

Scalable Big Data platforms are used by **Google, Amazon, Netflix and banks** for recommendation engines, log analysis and fraud detection on petabytes of data.

Unit 2: Advanced Data Analysis Techniques

Q1. Short Definitions (2 Marks Each)

(a) What is Principal Component Analysis (PCA)?

- **PCA** is an **unsupervised dimensionality-reduction technique**. It transforms a set of possibly correlated variables into a smaller set of **uncorrelated variables called principal components**, ordered so that the **first component captures the maximum variance**.
- The components are the **eigenvectors of the covariance matrix**, and their **eigenvalues** give the variance each one explains.

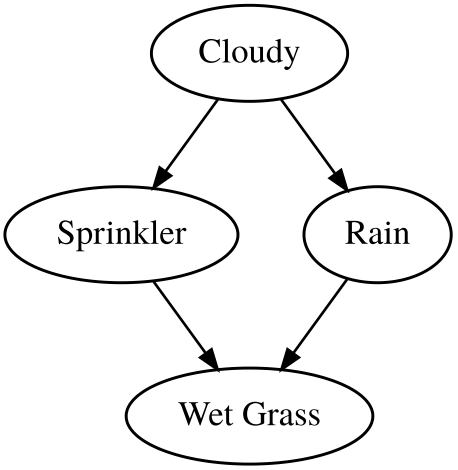
$$\text{Variance explained by } PC_k = \frac{\lambda_k}{\sum_i \lambda_i} \times 100\%$$

- *Uses:* reducing features before ML, visualising high-dimensional data in 2-D, noise reduction, image compression.

(b) Define a Bayesian Network.

- A **Bayesian Network (Belief Network)** is a **probabilistic graphical model**. It is a **Directed Acyclic Graph (DAG)** in which **nodes are random variables** and **edges show conditional dependencies**. Each node has a **Conditional Probability Table (CPT)** giving $P(\text{node} \mid \text{parents})$.
- The joint probability factorises as:

$$P(X_1, X_2, \ldots, X_n) = \prod_{i=1}^{n} P(X_i \mid \text{Parents}(X_i))$$



Example Bayesian network

(c) What is the core concept behind Fuzzy Logic?

- **Fuzzy logic** works with **degrees of truth**, between 0 and 1, instead of the crisp **true or false** of Boolean logic. An element belongs to a **fuzzy set** with a **membership value** $\mu(x) \in [0, 1]$.
- *Example:* a temperature of 30°C may be "Hot" with $\mu = 0.7$ and "Warm" with $\mu = 0.3$.
- It reasons with **IF-THEN rules** on linguistic variables, through **fuzzification, rule evaluation and defuzzification**. Used in washing machines, AC controllers and auto-focus cameras.

Q2. Explain Regression Modeling and Multivariate Analysis. (7 Marks)

1. Introduction / Definition

Regression modeling is a statistical technique that estimates the **relationship between a dependent (target) variable Y and one or more independent (predictor) variables X**, so that Y can be **predicted**. **Multivariate analysis** is a set of techniques that **analyse more than one variable at the same time**, to understand the relationships, patterns and structure among them.

2. Block Diagram



Types of regression and multivariate techniques

3. Core Explanation / Working

(a) Regression modeling

Simple linear regression: $\widehat{Y} = b_0 + b_1 X$. The coefficients come from the **least squares** method:

$$b_1 = \frac{\sum \left(X_i - \bar{X}\right)\left(Y_i - \bar{Y}\right)}{\sum \left(X_i - \bar{X}\right)^2}, \qquad b_0 = \bar{Y} - b_1 \bar{X}$$

Multiple linear regression: $\widehat{Y} = b_0 + b_1X_1 + b_2X_2 + \ldots + b_nX_n$. In matrix form the solution is:

$$\mathbf{b} = \left(X^TX\right)^{-1}X^TY$$

Logistic regression: used when Y is binary (0 or 1):

$$P\left(Y = 1\right) = \frac{1}{1 + e^{-\left(b_0 + b_1X_1 + \ldots + b_nX_n\right)}}$$

Steps of regression modeling

1. Collect the data and choose the dependent and independent variables.
2. Check the **assumptions**: linearity, independence of errors, **homoscedasticity** (constant variance), normality of residuals, and no multicollinearity.
3. **Estimate the coefficients** (by least squares or maximum likelihood).
4. **Evaluate the fit** with R^2, adjusted R^2, RMSE and p-values:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum\left(Y_i - \widehat{Y}_i\right)^2}{\sum\left(Y_i - \bar{Y}\right)^2}$$

5. **Predict** new values and interpret the coefficients.

(b) Multivariate analysis

- **Multiple regression:** one dependent variable with many predictors.
- **Principal Component Analysis / Factor Analysis:** reduces many correlated variables to a few components or factors.
- **Cluster analysis:** groups similar observations (k-means, hierarchical).
- **Discriminant analysis:** classifies observations into predefined groups.
- **MANOVA:** compares the means of **several dependent variables** across groups.
- **Canonical correlation:** studies the relationship between two **sets** of variables.

4. Key Characteristics - Univariate vs. Bivariate vs. Multivariate

Basis	Univariate	Bivariate	Multivariate
Variables	One	Two	**Three or more**
Purpose	Describe a distribution	Find the relationship between two variables	Find complex relationships and patterns
Techniques	Mean, SD, histogram	Correlation, simple regression, scatter plot	Multiple regression, PCA, clustering, MANOVA
Example	Average salary	Height vs. weight	Price depending on area, location, age and rooms

5. Advantages & Disadvantages

Advantages

- Regression gives **quantitative predictions** and shows the **strength of each predictor**.
- Multivariate analysis captures **real-world situations**, where many factors act together.

- Both support **data-driven decisions** (pricing, risk, forecasting).

Disadvantages

- They need **assumptions** (linearity, normality), and the results can mislead if these are violated.
- **Multicollinearity** and **outliers** distort the coefficients. Results are harder to interpret as the number of variables grows.

6. Conclusion / Application

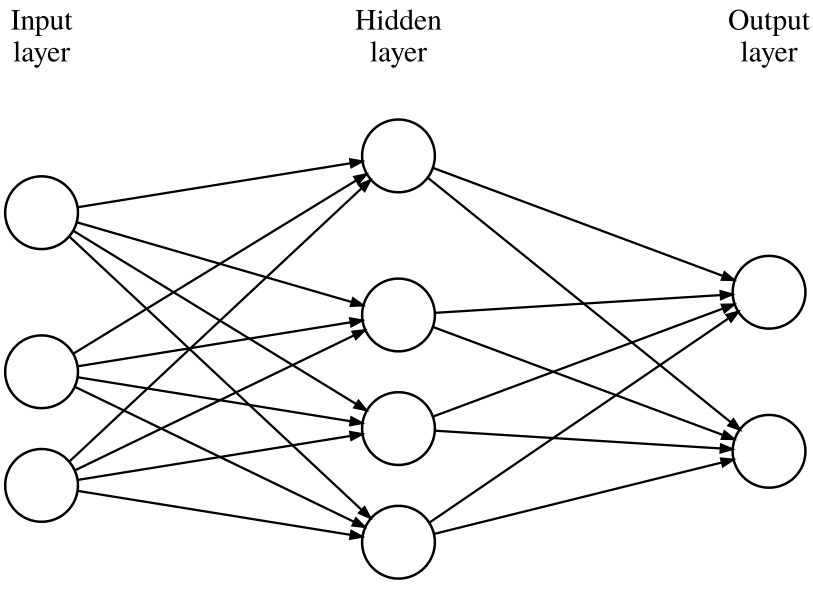
Regression and multivariate analysis are used for **sales forecasting, house-price prediction, credit-risk scoring and market segmentation**.

Q3. Discuss the architecture of Neural Networks, focusing on competitive learning and generalization. (7 Marks)

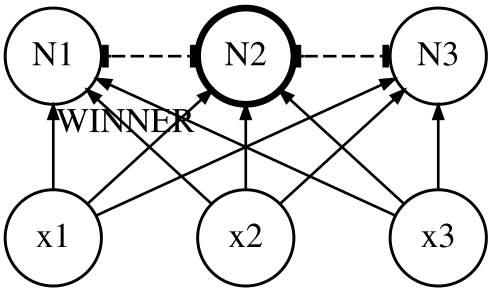
1. Introduction / Definition

An **Artificial Neural Network (ANN)** is a computing model inspired by the brain. It consists of **layers of interconnected neurons**, and each neuron computes a **weighted sum of its inputs followed by an activation function**. **Competitive learning** is an **unsupervised** scheme in which neurons **compete** to respond to an input, and **only the winner learns**. **Generalization** is the network's ability to give **correct outputs for unseen data**.

2. Block Diagram



Feed-forward neural network architecture



Competitive (winner-take-all) learning layer

(Dashed lines are lateral inhibition: the output neurons suppress each other.)

3. Core Explanation / Working

(a) Architecture of a neural network

- **Neuron model:** $y = f\left(\sum_i w_i x_i + b \right)$, where f is an activation function (sigmoid, tanh, ReLU).
- **Input layer:** receives the features. **Hidden layers:** learn intermediate representations. **Output layer:** gives the prediction.
- **Types of architecture:**
  - **Feed-forward** (MLP): signals move in one direction only.
  - **Recurrent** (RNN, LSTM): has feedback loops, which give memory for sequences.
  - **Competitive / self-organising** (Kohonen SOM): a single layer of competing neurons.
- **Learning:** supervised (backpropagation), unsupervised (competitive or Hebbian) or reinforcement.

(b) Competitive learning

- The output neurons **compete**. The neuron whose weight vector is **closest to the input** wins. This is "**winner-take-all**".
- **Winner selection:** $j^* = \arg\min_j \| \mathbf{x} - \mathbf{w}_j \|$
- **Update (only the winner):** $\mathbf{w}_{j^*} \leftarrow \mathbf{w}_{j^*} + \eta \left(\mathbf{x} - \mathbf{w}_{j^*} \right)$. This moves the winner **towards the input**.
- **Result:** each neuron becomes a **prototype (cluster centre)** for a group of similar inputs. It is an unsupervised way of **clustering**.
- **Kohonen Self-Organizing Map (SOM):** the winner **and its neighbours** are updated, which keeps the **topology** of the data on a 2-D map.
- **Problem:** "dead neurons" that never win. The fix is a **conscience mechanism** (frequent winners are penalised).

(c) Generalization

- A network **generalizes** well when it performs well on **new, unseen data**, not just on the training data.
- **Overfitting:** the network memorizes the noise (too many weights, too long training). **Underfitting:** the network is too simple.
- **Techniques to improve generalization:**
  - **Cross-validation** and **early stopping** (stop when the validation error starts rising).
  - **Regularization** (L1/L2 weight decay) and **dropout**.
  - **More training data** and data augmentation.
  - **Choosing a suitable network size** (number of hidden units).

4. Key Characteristics - Competitive Learning vs. Error-Correction (Backpropagation) Learning

Basis	Competitive Learning	Backpropagation Learning
Type	**Unsupervised**	**Supervised**
Target output	Not needed	Needed
Which weights change	**Only the winner's** (and its neighbours in SOM)	**All** weights
Rule	$\Delta w = \eta\,(x - w)$	$\Delta w = \eta\,\delta\,x$
Result	Clusters / prototypes	Input-output mapping
Example network	Kohonen SOM, LVQ	MLP

5. Advantages & Disadvantages

Advantages

- Neural networks **learn complex non-linear patterns** automatically.
- Competitive learning **discovers clusters without labels**.
- Good generalization gives **robust predictions** on new data and tolerance to noise.

Disadvantages

- They are **black boxes**, hard to interpret.
- They need **large data** and computing power. Competitive learning can leave dead neurons.
- They are prone to **overfitting** if generalization is not controlled.

6. Conclusion / Application

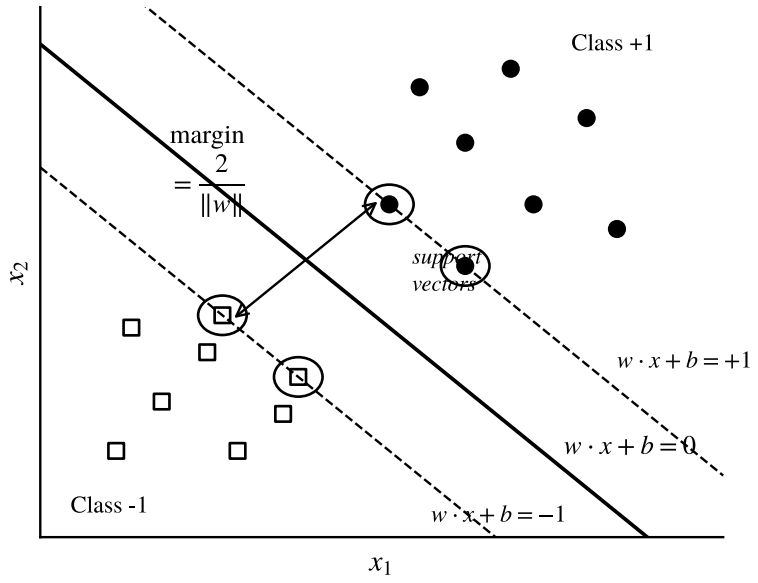
Neural networks are used for **image and speech recognition and fraud detection**, while competitive learning (SOM) is used for **customer segmentation, document clustering and data visualization**.

Q4. Explain Support Vector Machines (SVM) and kernel methods for data classification. (7 Marks)

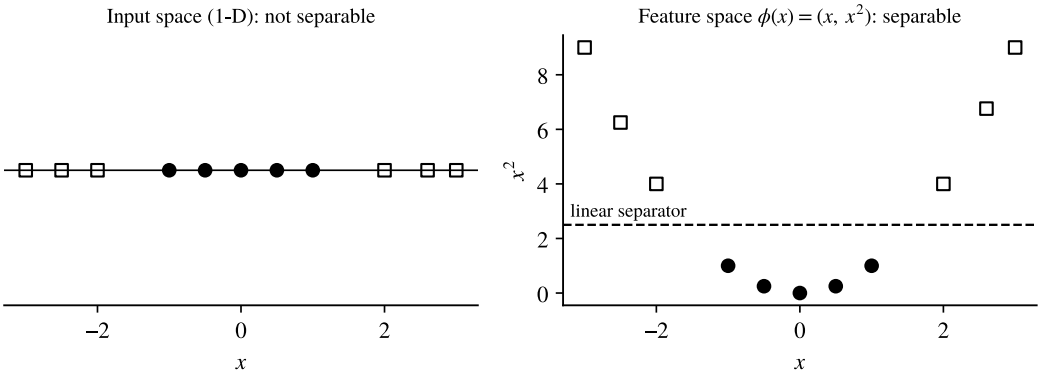
1. Introduction / Definition

A **Support Vector Machine (SVM)** is a supervised classifier that finds the **optimal hyperplane** separating two classes with the **maximum margin**. **Kernel methods** let an SVM classify **non-linearly separable data**. They implicitly map the data into a **higher-dimensional feature space**, where a linear separator exists, without ever computing that mapping (the **kernel trick**).

2. Block Diagram



Maximum-margin hyperplane with support vectors



Kernel trick: data not separable in 1-D becomes separable after mapping x to (x, x²)

3. Core Explanation / Working

(a) Linear SVM

- Hyperplane: $\mathbf{w} \cdot \mathbf{x} + b = 0$. Classification rule: $f(\mathbf{x}) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$.
- **Margin** = distance between the two boundary hyperplanes $\mathbf{w} \cdot \mathbf{x} + b = \pm 1$:

$$\text{Margin} = \frac{2}{\| \mathbf{w} \|}$$

- **Optimization (hard margin):**

$$\min_{\mathbf{w}, b} \frac{1}{2} \| \mathbf{w} \|^2 \quad \text{subject to} \quad y_i (\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 \quad \forall i$$

- The points lying on the margin are the **support vectors**. Only they decide the hyperplane.

(b) Soft-margin SVM (for noisy or overlapping data)

$$\min \frac{1}{2} \| \mathbf{w} \|^2 + C \sum_i \xi_i \quad \text{subject to} \quad y_i (\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

- **C** trades margin width against training errors. A large C gives a narrow margin and fewer errors.

(c) Kernel methods

- Map $\mathbf{x} \rightarrow \phi(\mathbf{x})$ into a higher-dimensional space. The SVM only needs **dot products**, so we use a kernel function:

$$K\left(\mathbf{x}_i, \mathbf{x}_j\right) = \phi(\mathbf{x}_i) \cdot \phi\left(\mathbf{x}_j\right)$$

- **Decision function:** $f(\mathbf{x}) = \text{sign}\left(\sum_{i \in SV} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b\right)$

Kernel	Formula	Use
Linear	$K = \mathbf{x}_i \cdot \mathbf{x}_j$	Linearly separable data, text
Polynomial	$K = \left(\mathbf{x}_i \cdot \mathbf{x}_j + c\right)^d$	Curved boundaries, image features
RBF (Gaussian)	$K = e^{-\gamma \parallel \mathbf{x}_i - \mathbf{x}_j \parallel^2}$	**Default choice**, complex boundaries
Sigmoid	$K = \tanh\left(\kappa\, \mathbf{x}_i \cdot \mathbf{x}_j + c\right)$	Behaves like a neural network

(d) Steps for SVM classification

1. **Scale / normalise** the features.
2. Choose a **kernel** (start with RBF).
3. Tune **C and** γ by grid search with cross-validation.
4. Train, find the support vectors, and classify new points.
5. For **multi-class** problems, use **One-vs-One** or **One-vs-Rest**.

4. Key Characteristics - Linear SVM vs. Kernel SVM

Basis	Linear SVM	Kernel (Non-linear) SVM
Decision boundary	Straight line / hyperplane	Curved boundary
Data	Linearly separable	Non-linearly separable
Training speed	Fast	Slower
Parameters	C	C and kernel parameters (γ, d)
Overfitting risk	Low	Higher (if γ is large)

5. Advantages & Disadvantages

Advantages

- **Effective in high dimensions**, and memory efficient (only the support vectors are stored).
- The **kernel trick** handles complex boundaries. Maximum margin gives **good generalization**.

Disadvantages

- **Slow on very large datasets**, and **sensitive** to the choice of kernel and parameters.
- Gives **no direct probability** output, and the model is hard to interpret.

6. Conclusion / Application

SVMs with kernels are used for **text and spam classification, face detection, handwriting recognition and bioinformatics** (gene classification).

Unit 3: Mining Data Streams

Q1. Short Definitions (2 Marks Each)

(a) What defines a Data Stream?

- A **data stream** is a **continuous, unbounded, rapidly arriving and time-ordered sequence of data elements**. It can be read **only once, or a few times**, in a single pass, using **limited memory**.
- **Characteristics:**
  - **Infinite length**, so the whole stream cannot be stored.
  - **High arrival rate**, so processing must be real-time.
  - **Concept drift:** the data distribution changes over time.
  - Answers are often **approximate**.
- *Examples:* sensor readings, stock ticks, click-streams, tweets, network packets.

(b) How is a decaying window different from a sliding window?

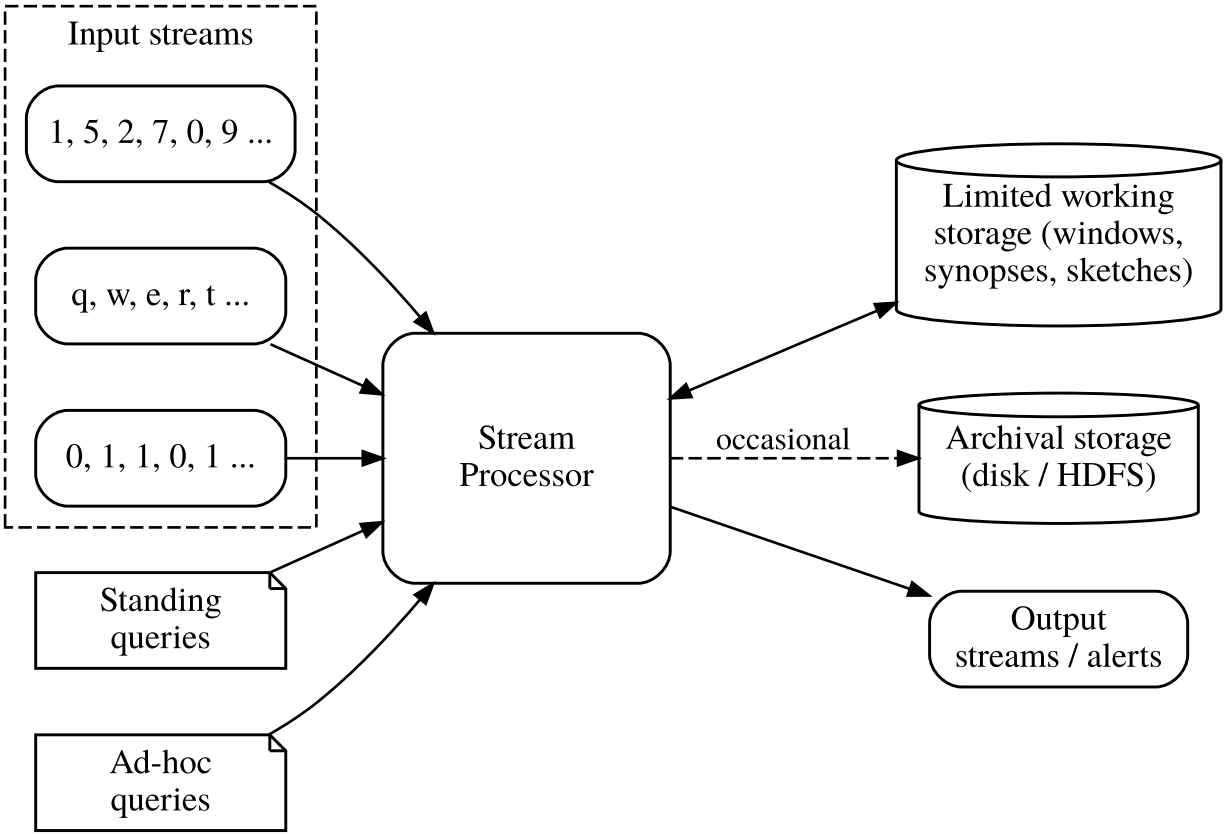
Sliding Window	Decaying Window
Considers only the **last N elements** (or the last T seconds)	Considers **all elements**, with **exponentially decreasing weights** for older ones
Old elements are **dropped completely** once outside the window	Old elements **fade gradually** and are never cut off sharply
Needs storage for the N elements	Needs only **one running value** per item
Update: add the new element, remove the oldest	Update: $S \leftarrow S\,(1-c) + a_t$
Equal weight inside the window	Weight of an element of age i is $(1-c)^i$

Q2. Explain the complete architecture of a Stream Computing model. (7 Marks)

1. Introduction / Definition

Stream computing is a paradigm that **processes data continuously as it arrives** (data in motion), instead of first storing it and querying it later (data at rest). A **Data Stream Management System (DSMS)** receives many input streams, keeps **limited working storage and summaries**, runs **standing (continuous) queries and ad-hoc queries**, and produces **output streams** in real time.

2. Block Diagram - Stream Data Model (DSMS)



Architecture of a data stream management system

3. Core Explanation / Working

(a) Components

- **Input streams:** many sources, each with its own rate and data type. Examples: sensors, logs, tweets, transactions.
- **Stream processor:** the core engine. It reads each element **once** and applies the queries and algorithms.
- **Working storage (main memory): limited**. It holds **windows, samples, synopses and sketches** (for example Bloom filters, FM sketches, histograms).
- **Archival storage:** large but slow. It holds the raw stream for offline analysis and is **not used for real-time answers**.
- **Standing (continuous) queries:** registered once and **evaluated continuously**. Example: “Alert if the temperature exceeds 50°C”.
- **Ad-hoc queries:** asked once about the current state. Example: “How many distinct users in the last hour?”.
- **Output streams:** results, alerts and dashboards produced in real time.

(b) Stream processing steps

1. **Ingestion:** collect the data through message brokers (Apache Kafka, Flume).
2. **Pre-processing:** filter, parse, clean and timestamp.
3. **Windowing:** split the infinite stream into **finite windows** (tumbling, sliding, decaying).
4. **Analysis:** sampling, filtering (Bloom filter), counting distinct elements (Flajolet-Martin), estimating moments (AMS), counting ones (DGIM).
5. **Output:** push the results to dashboards, alerts or a database.

(c) Key stream algorithms

- **Sampling:** keep a representative fraction (reservoir sampling).
- **Filtering:** a **Bloom filter** tests membership using k hash functions and a bit array.
- **Counting distinct elements: Flajolet-Martin**.
- **Estimating moments:** the **Alon-Matias-Szegedy (AMS)** algorithm.
- **Counting ones in a window:** the **DGIM** algorithm.

(d) Stream computing frameworks

- **Apache Storm, Apache Flink, Spark Streaming, Kafka Streams, IBM InfoSphere Streams**.

4. Key Characteristics - DBMS vs. DSMS

Basis	DBMS (Data at rest)	DSMS (Data in motion)
Data	Persistent, finite	Transient, **infinite**
Access	Random, many passes	**Sequential, one pass**
Queries	One-time (ad-hoc)	**Continuous (standing)**
Answers	Exact	Often **approximate**
Storage	Unlimited disk	**Limited main memory**
Latency	Seconds to hours	**Milliseconds**

5. Advantages & Disadvantages

Advantages

- Gives **real-time insights** and immediate reactions (fraud alerts, monitoring).
- **Low storage cost**, because only summaries are kept.
- **Scales** to very high data rates with distributed frameworks.

Disadvantages

- Answers are **approximate**, and the data cannot be revisited.
- Handling **out-of-order data, concept drift and failures** is complex.

6. Conclusion / Application

Stream computing is used for **credit-card fraud detection, network intrusion detection, IoT sensor monitoring, stock trading and live social-media analytics**.

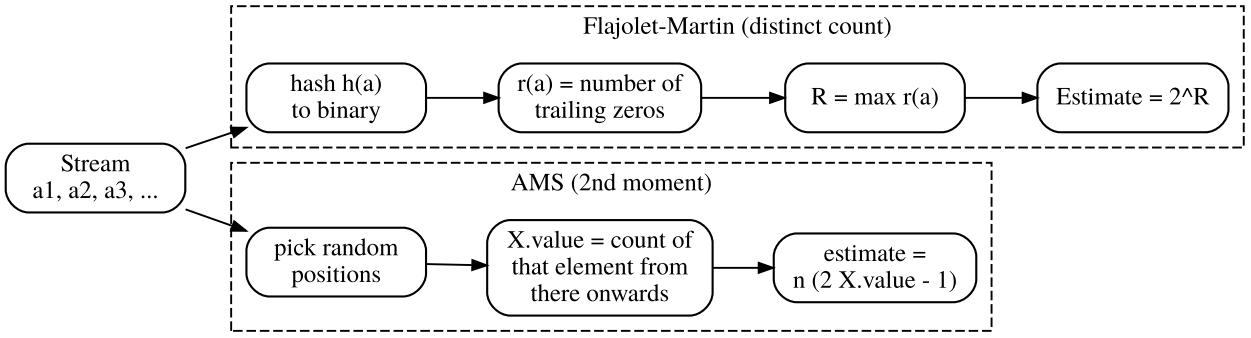
Q3. Explain the techniques for estimating moments and counting distinct elements in a stream. (7 Marks)

1. Introduction / Definition

In a stream we **cannot store every element**, so statistics must be **estimated** with small memory. **Counting distinct elements** (the 0th moment) is solved by the **Flajolet-Martin (FM) algorithm**, which uses hashing and trailing zeros. **Moments** in general, especially the **second moment (the "surprise number")**, are estimated with the **Alon-**

Matias-Szegedy (AMS) algorithm.

2. Block Diagram



Flajolet-Martin and AMS: one pass, small memory

3. Core Explanation / Working

(a) Moments of a stream

- If element i occurs m_i times, the **k-th moment** is:

$$F_k = \sum_i (m_i)^k$$

- F_0 = number of **distinct** elements.
- F_1 = **length** of the stream.
- F_2 = the **surprise number**, a measure of how uneven the distribution is.

(b) Flajolet-Martin algorithm (counting distinct elements, F0)

1. Choose a hash function h that maps each element to a binary string of enough bits.
2. For each element a, compute h(a). Let **r(a)** be the **number of trailing zeros** in h(a).
3. Keep **R = the maximum r(a)** seen so far.
4. **Estimate:** number of distinct elements $\approx 2^R$.

- **Why it works:** the probability that a hash value ends in at least r zeros is 2^{-r}. To see r trailing zeros, we must have seen about 2^r distinct elements.
- **Improving accuracy:** use **many hash functions**, take the **average inside groups**, then the **median of the group averages**.
- *Example:* if $h(a) = 110000$ has 2 trailing zeros and that is the largest seen, the estimate is $2^2 = 4$ distinct elements.

(c) AMS algorithm (estimating the second moment, F2)

1. Choose some **random positions** in the stream (stream length n).
2. For each chosen position t with element a, keep a variable X with **X.element = a** and **X.value = the number of times a appears from position t onwards**.
3. **Estimate:**

$$F_2 \approx n\,(2X.\text{value} - 1)$$

4. Average over many variables X for accuracy.

- *Example:* stream a, b, c, b, d, a, c, d, a, b, d, c, a, a, b (n = 15). Here a = 5, b = 4, c = 3, d = 3, so the true $F_2 = 25 + 16 + 9 + 9 = 59$. If the variable at position 3 (element c) counts 3 occurrences of c from there on, the estimate is $15\,(2 \times 3 - 1) = 75$. Averaging many such estimates approaches 59.

(d) Higher moments ($k > 2$): use $n\left(v^k - (v-1)^k\right)$, where v = X.value.

4. Key Characteristics - FM vs. AMS

Basis	Flajolet-Martin	AMS
Estimates	F_0 (distinct count)	F_2 (and higher moments)
Main idea	Trailing zeros of hash values	Counts from random positions
Memory	$O(\log n)$ bits per hash	One counter per variable
Formula	2^R	$n\,(2X.\text{value} - 1)$
Accuracy improvement	Average + median of many hashes	Average of many variables

5. Advantages & Disadvantages

Advantages

- **Single pass** and **very little memory**, independent of the stream size.
- Gives a **fast, scalable approximation** for huge streams.

Disadvantages

- Results are **approximate**, and a single hash function gives high variance.
- Needs **good, independent hash functions**, and accuracy depends on the number of estimators used.

6. Conclusion / Application

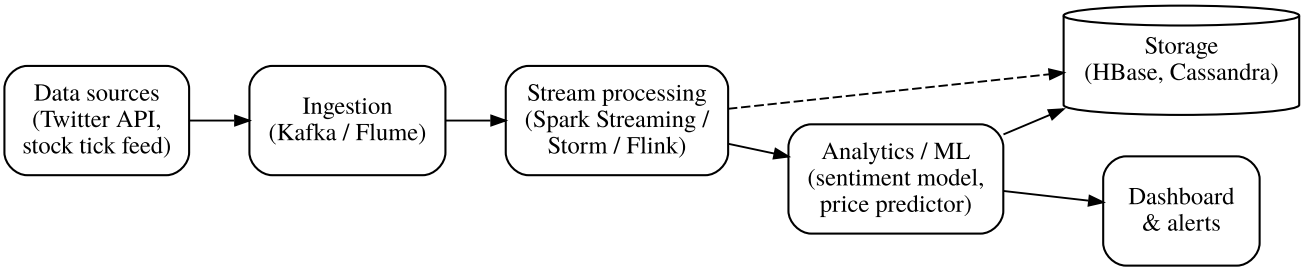
These techniques count **unique visitors to a website, distinct IP addresses in network traffic and unique search queries** (HyperLogLog, used in Redis and Google BigQuery, is an improved FM).

Q4. Discuss Real-Time Analytics Platform (RTAP) applications, with sentiment analysis or stock market prediction as a case study. (7 Marks)

1. Introduction / Definition

A **Real-Time Analytics Platform (RTAP) collects, processes and analyses data streams as they arrive**, so that **decisions can be made within milliseconds to seconds**. It combines **stream ingestion, stream processing, analytics or ML models and visualization**.

2. Block Diagram - RTAP for Twitter Sentiment / Stock Prediction



Real-time analytics pipeline

3. Core Explanation / Working

(a) Components of an RTAP

1. **Data sources:** social media, sensors, market feeds, logs.
2. **Ingestion layer: Apache Kafka, Flume**. It buffers and distributes the streams.
3. **Processing layer: Spark Streaming, Apache Storm, Flink**. It runs windowed computations.
4. **Analytics layer:** ML models, rules, complex event processing.
5. **Storage layer: NoSQL (HBase, Cassandra)** for fast reads and writes, and HDFS for archives.
6. **Presentation layer:** dashboards (Kibana, Grafana), alerts, APIs.

(b) Case study 1: Real-time sentiment analysis of tweets

1. **Collect** tweets that contain a brand or hashtag (for example "#iPhone") using the Twitter streaming API, then send them to Kafka.
2. **Pre-process:** remove URLs, emojis and stop-words, then tokenise and stem.
3. **Classify sentiment** (positive, negative or neutral):
    - **Lexicon-based:** a score from word lists (VADER, SentiWordNet).
    - **ML-based:** Naive Bayes, SVM or LSTM trained on labelled tweets.
4. **Aggregate** in a **sliding window** (for example the % of positive tweets in the last 5 minutes).
5. **Visualize and alert:** a live sentiment graph. If negative sentiment spikes, the PR team is alerted.

- **Use:** brand monitoring, election trend analysis, product-launch feedback.

(c) Case study 2: Stock market prediction

1. **Ingest** live price ticks, trading volume, news headlines and social sentiment.
2. **Compute features** over sliding windows: moving averages (SMA, EMA), RSI, volatility, sentiment score.
3. **Predict** price movement with models like **ARIMA, LSTM or regression**.
4. **Act:** trigger buy or sell signals (algorithmic trading) or risk alerts, within **milliseconds**.

- **Challenges:** very high velocity, noisy data, the need for ultra-low latency, and regulatory compliance.

(d) Other RTAP applications

- **Fraud detection** in banking (flag suspicious card transactions instantly).
- **IoT and predictive maintenance** (machine sensor alerts).
- **Network security** (intrusion detection).
- **E-commerce recommendations** and dynamic pricing.
- **Traffic management** and ride-hailing (Uber surge pricing).

4. Key Characteristics - Batch Analytics vs. Real-Time Analytics

Basis	Batch Analytics	Real-Time Analytics (RTAP)
Data	Stored, historical	**Live streams**
Latency	Minutes to hours	**Milliseconds to seconds**
Tools	Hadoop MapReduce, Hive	Kafka, Storm, Spark Streaming, Flink
Use case	Monthly reports, model training	Fraud alerts, live sentiment, trading
Processing	Whole dataset	Windows of recent data

5. Advantages & Disadvantages

Advantages

- **Immediate decisions**, a competitive edge and faster responses to events.
- **Detects anomalies as they happen**, which prevents losses (fraud).
- Gives a better **customer experience** through live personalisation.

Disadvantages

- **Complex, costly** infrastructure that needs fault-tolerant, scalable systems.
- **Data quality** and noise (sarcasm in tweets) reduce accuracy. Low-latency ML is hard.

6. Conclusion / Application

RTAPs power **Twitter trend detection, Bloomberg trading terminals, PayPal fraud systems and Uber's real-time dispatch**.

Unit 4: Frequent Itemsets & Clustering

Q1. Short Definitions (2 Marks Each)

(a) What is the Apriori property?

- **Apriori property (downward closure / anti-monotone property):** *"Every non-empty subset of a frequent itemset must also be frequent."* Equivalently, **if an itemset is infrequent, all of its supersets are also infrequent**.
- **Use:** it lets the Apriori algorithm **prune** candidate itemsets. For example, if {Bread, Jam} is infrequent, then {Bread, Jam, Milk} need not be counted at all.

$$\text{If } X \subseteq Y, \text{ then } \text{support}(Y) \leq \text{support}(X)$$

(b) Differentiate between K-Means and Hierarchical clustering.

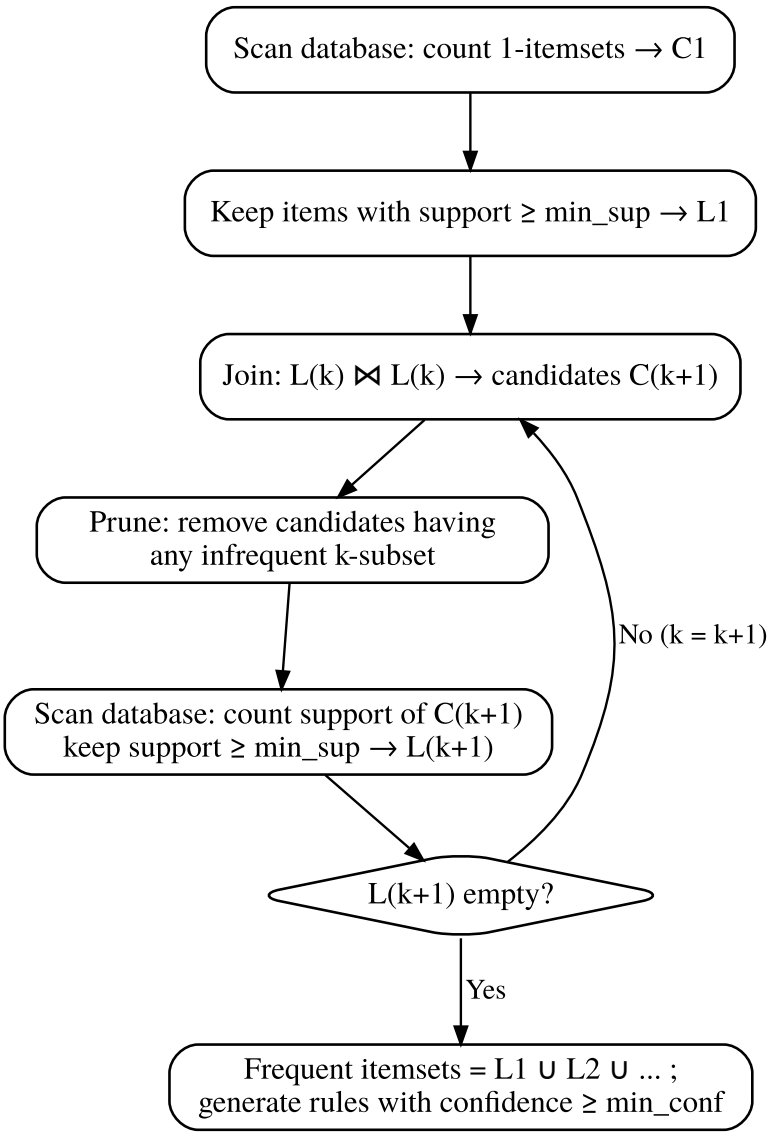
Basis	K-Means	Hierarchical
Type	**Partitional** (flat clusters)	**Tree-based** (dendrogram)
Number of clusters	**K must be given** in advance	Not needed; cut the dendrogram at any level
Approach	Iterative re-assignment to centroids	Agglomerative (bottom-up) or divisive (top-down) merging or splitting
Complexity	$O(nkt)$, fast	$O\left(n^2\right)$ to $O\left(n^3\right)$, slow
Large datasets	Suitable	Not suitable
Result	Depends on the initial centroids	Deterministic
Cluster shape	Spherical	Any shape (depends on the linkage)

Q2. Trace the Apriori algorithm step by step on a transactional dataset to find frequent itemsets. (7 Marks)

1. Introduction / Definition

The **Apriori algorithm** (Agrawal and Srikant, 1994) finds **frequent itemsets** in a transaction database, which are itemsets whose **support ≥ minimum support**. It then generates **association rules**. It works **level by level**: frequent k-itemsets L_k are used to generate the candidate (k+1)-itemsets C_{k+1}, and the **Apriori property** prunes candidates that have an infrequent subset.

2. Flowchart



Apriori algorithm

3. Core Explanation / Working

Formulas

$$\text{Support}(X) = \frac{\text{Number of transactions containing } X}{\text{Total transactions}}, \qquad \text{Confidence}(X \Rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

Example dataset (minimum support count = 2, i.e. 2/9 = 22%)

TID	Items
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

Step 1: C1 and L1 (count every single item)

Itemset	{I1}	{I2}	{I3}	{I4}	{I5}
Support count	6	7	6	2	2

- All counts are ≥ 2, so **L1 = {I1}, {I2}, {I3}, {I4}, {I5}**.

Step 2: C2 = L1 ⋈ L1, then count

Itemset	{I1,I2}	{I1,I3}	{I1,I4}	{I1,I5}	{I2,I3}	{I2,I4}	{I2,I5}	{I3,I4}	{I3,I5}	{I4,I5}
Count	4	4	1	2	4	2	2	0	1	0

- **L2 = {I1,I2}, {I1,I3}, {I1,I5}, {I2,I3}, {I2,I4}, {I2,I5}**. ({I1,I4}, {I3,I4}, {I3,I5} and {I4,I5} are removed because their count is below 2.)

Step 3: C3 = L2 ⋈ L2, prune, then count

Candidate	2-subsets	All subsets in L2?	Count	In L3?
{I1, I2, I3}	{I1,I2}, {I1,I3}, {I2,I3}	Yes	**2**	**Yes**
{I1, I2, I5}	{I1,I2}, {I1,I5}, {I2,I5}	Yes	**2**	**Yes**
{I1, I3, I5}	{I3,I5} is not in L2	**No, pruned**	-	No
{I2, I3, I4}	{I3,I4} is not in L2	**No, pruned**	-	No
{I2, I3, I5}	{I3,I5} is not in L2	**No, pruned**	-	No
{I2, I4, I5}	{I4,I5} is not in L2	**No, pruned**	-	No

- **L3 = {I1, I2, I3}, {I1, I2, I5}**

*Step 4: C4 = L3 ⋈ L3 gives {I1, I2, I3, I5}. Its subset {I2, I3, I5} is not frequent, so it is **pruned**. C4 is empty, so the algorithm **stops**.*

Step 5: Association rules from {I1, I2, I5} (support = 2/9 = 22%)

Rule	Confidence	Strong if min_conf = 70%?
I1 ∧ I2 ⇒ I5	2/4 = 50%	No
I1 ∧ I5 ⇒ I2	2/2 = **100%**	**Yes**
I2 ∧ I5 ⇒ I1	2/2 = **100%**	**Yes**
I1 ⇒ I2 ∧ I5	2/6 = 33%	No
I2 ⇒ I1 ∧ I5	2/7 = 29%	No
I5 ⇒ I1 ∧ I2	2/2 = **100%**	**Yes**

(More Apriori numericals, including the letter and grocery datasets from PYQs, are solved in the Numerical Problems section of this subject.)

4. Key Characteristics - Apriori vs. FP-Growth

Basis	Apriori	FP-Growth
Candidate generation	**Yes** (join + prune)	**No**
Database scans	k + 1 scans (one per level)	**2 scans**
Data structure	Hash tree of candidates	**FP-tree** (compressed prefix tree)
Speed	Slow on dense, large data	**Faster**
Memory	Many candidates	FP-tree must fit in memory

5. Advantages & Disadvantages

Advantages

- **Simple and easy to understand**. The Apriori property gives **effective pruning**.
- Finds **all** frequent itemsets and is easily parallelised.

Disadvantages

- **Many database scans**, and a **huge number of candidates** when min_sup is low.
- **Slow** on large, dense datasets.

6. Conclusion / Application

Apriori is used for **market-basket analysis** (store layout, “frequently bought together”), **cross-selling, medical diagnosis (co-occurring symptoms) and web usage mining**.

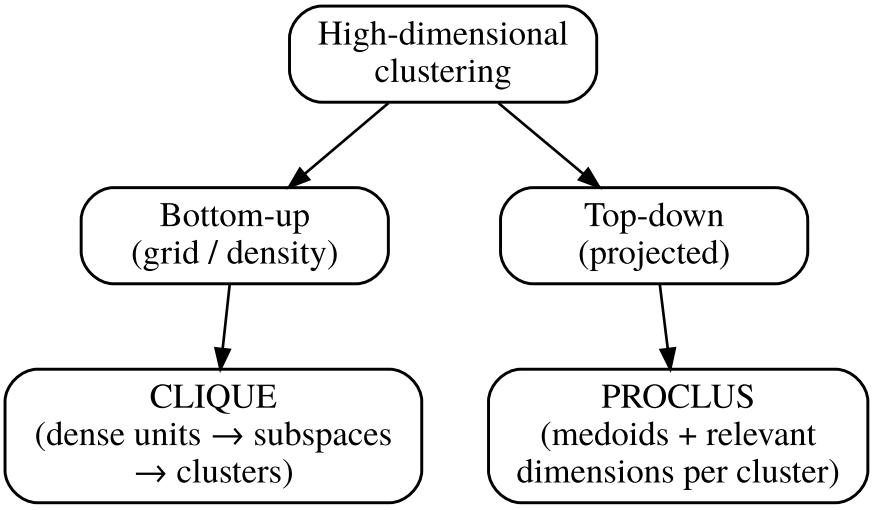
Q3. Explain the CLIQUE and PROCLUS algorithms for clustering high-dimensional data. (7 Marks)

1. Introduction / Definition

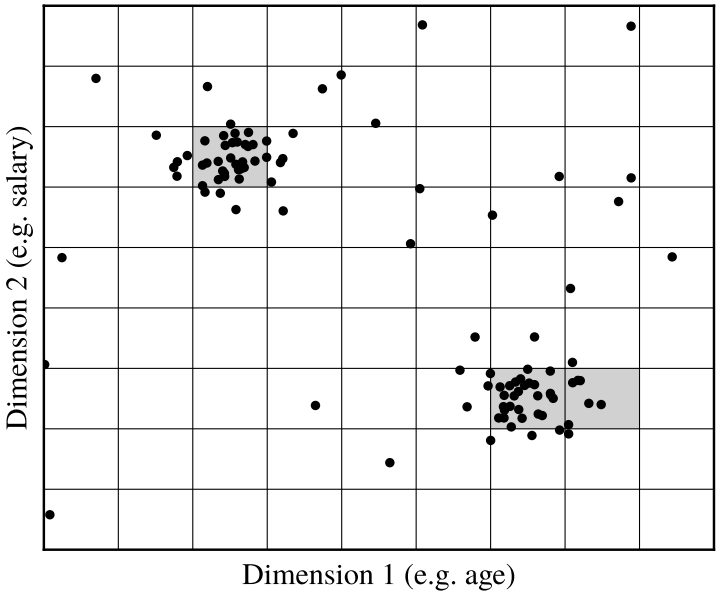
In **high-dimensional data** (many attributes), distances become meaningless (the **curse of dimensionality**), and clusters often exist **only in subsets of the dimensions (subspaces)**. **CLIQUE** (CLustering In QUEst) is a **grid- and density-based subspace clustering** algorithm that works **bottom-up**. **PROCLUS** (PROjected CLUStering) is a

top-down, k-medoid-based projected clustering algorithm that finds, for each cluster, its **own subset of relevant dimensions**.

2. Block Diagram



Subspace clustering approaches



CLIQUE: dense grid units (shaded) in 2-D; dense units in both 1-D projections suggest candidate dense cells

3. Core Explanation / Working

(a) CLIQUE

1. **Partition** each dimension into ξ **equal-width intervals**. This gives a grid of **units (cells)**.
2. A unit is **dense** if the fraction of points in it is greater than a **density threshold** τ.
3. **Bottom-up search (Apriori-like):**
    - Find the dense units in **each 1-D subspace**.
    - Generate the candidate dense units in **k-dimensional** subspaces from the dense (k−1)-dimensional ones. Use the **monotonicity property**: *if a k-dimensional unit is dense, its projections in every (k−1)-dimensional subspace are also dense.*

4. **Find clusters:** a cluster is a set of **connected dense units** (sharing a face) within a subspace.
5. **Generate a minimal description** of each cluster as a **DNF expression** of maximal regions (rectangles). Example: $(30 \leq \text{age} < 50) \wedge (4 \leq \text{salary} < 8)$.

- **Features:**
  - Automatically finds the **subspaces** that contain clusters.
  - **Insensitive to the order of input**, and gives **interpretable** results.
  - **Scales linearly** with the number of points.
- **Limitations:** accuracy depends on the **grid size ξ and threshold τ**. Using one density threshold for all dimensionalities can miss clusters.

(b) PROCLUS

- It is a **k-medoid-like** algorithm that finds **k clusters**, each in its **own projected subspace** of average dimensionality **l** (both given by the user).

- **Phase 1: Initialization.** Choose a **random sample** of points, then use a **greedy method** to pick a set of **well-separated potential medoids**.

- **Phase 2: Iterative phase.**

  1. Pick **k medoids** from the candidates.
  2. For each medoid, find its **locality** (the points near it). Choose the **dimensions along which the locality points are closest** to the medoid (smallest average distance). These become that cluster's **relevant dimensions**.
  3. **Assign each point** to the nearest medoid, using the **Manhattan segmental distance** over that medoid's dimensions only.
  4. **Replace bad medoids** (those with very small clusters) and repeat while the quality improves.

- **Phase 3: Refinement.** Recompute the dimensions using the final clusters, re-assign the points, and mark distant points as **outliers**.

- **Features:**

  - **Faster** than CLIQUE.
  - Gives **non-overlapping** clusters with their own dimensions.
  - Handles **outliers**.

- **Limitations:** the user must give **k and l**, and the results depend on the random initialization.

4. Key Characteristics - CLIQUE vs. PROCLUS

Basis	CLIQUE	PROCLUS
Approach	**Bottom-up**, grid- and density-based	**Top-down**, partition (k-medoid) based
Input parameters	Grid size ξ, density threshold τ	Number of clusters k, average dimensions l
Cluster overlap	Clusters **may overlap** (a point can be in several subspaces)	**Disjoint** clusters
Cluster shape	Arbitrary (union of dense cells)	Roughly spherical in their subspace
Outliers	Points in sparse cells	Explicitly detected
Output	Clusters + DNF description	Clusters + relevant dimensions + outliers
Speed	Exponential in the subspace dimensionality	**Faster**

5. Advantages & Disadvantages

Advantages

- Both **handle high-dimensional data** by finding clusters in **subspaces**.
- CLIQUE gives **interpretable** rules and finds clusters of **arbitrary shape**. PROCLUS is **efficient** and detects **outliers**.

Disadvantages

- CLIQUE is **sensitive to the grid size and threshold**. PROCLUS needs **k and l in advance**.
- Both may **miss** clusters in very sparse or oddly shaped subspaces.

6. Conclusion / Application

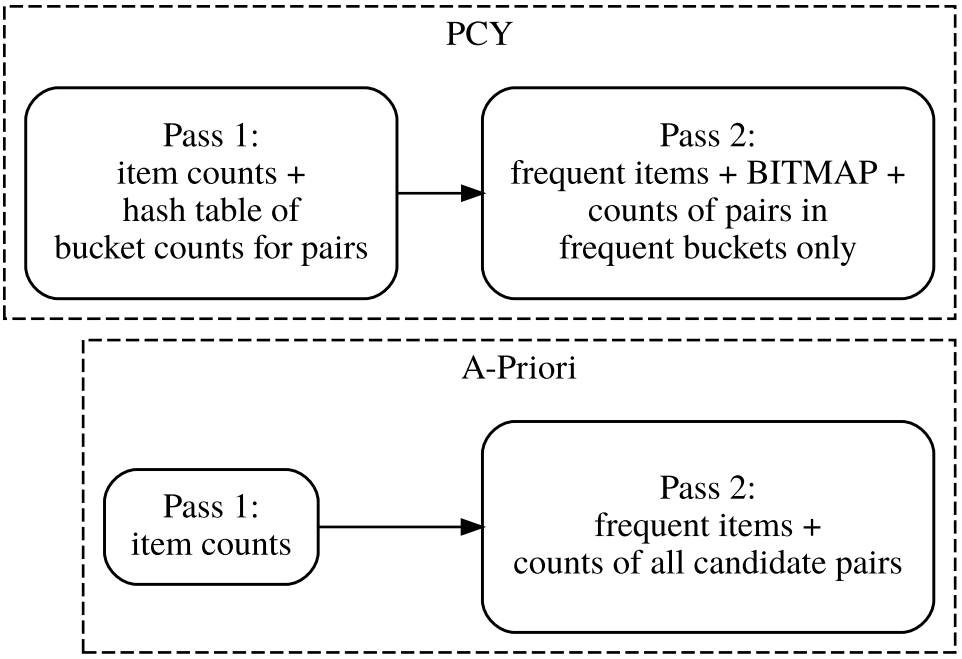
These algorithms are used on **gene-expression data, text documents, customer data with hundreds of attributes and image feature vectors**.

Q4. Discuss how to handle large datasets in main memory using limited-pass algorithms. (7 Marks)

1. Introduction / Definition

When transaction data is **too large for main memory**, frequent-itemset mining must **reduce the number of passes over the disk** and **use memory efficiently**. **Limited-pass algorithms**, namely **PCY (Park-Chen-Yu), Multistage, Multihash, the Random Sampling (Simple) algorithm, SON (Savasere-Omiecinski-Navathe) and Toivonen's algorithm**, find frequent itemsets in **two passes or fewer**, or with sampling.

2. Block Diagram



Main-memory use in A-Priori vs. PCY

3. Core Explanation / Working

(a) The problem

- In A-Priori, the **second pass (counting pairs)** needs the most memory, because the number of candidate pairs is about $\binom{n}{2}$.
- The goal is **fewer candidate pairs** and **fewer passes**.

(b) PCY algorithm (Park-Chen-Yu)

- **Pass 1:** count the single items. **In the spare memory**, hash **every pair** in each basket into a **hash table of buckets** and increment that bucket's count.
- **Between the passes:** replace the bucket counts with a **bitmap**, where 1 means the bucket count ≥ support (a **frequent bucket**). This uses only 1/32 of the memory.
- **Pass 2:** count pair {i, j} **only if** (1) both i and j are frequent, **and** (2) {i, j} hashes to a **frequent bucket**.
- **Benefit:** many infrequent pairs are eliminated, so pass 2 uses much less memory.

(c) Multistage algorithm

- Adds **extra intermediate passes**. In pass 2, it re-hashes only the pairs that qualified in pass 1, using a **second hash function**. Pass 3 counts only pairs that hash to frequent buckets in **both** tables.
- This gives **fewer false positives**, at the cost of extra passes.

(d) Multihash algorithm

- Uses **two or more hash tables in the same first pass**. A pair is counted in pass 2 only if it falls in frequent buckets in **all** the tables.

(e) Random sampling (Simple) algorithm

- Take a **random sample** of baskets that fits in memory. Run A-Priori **in main memory** with a **proportionally lowered support threshold** (for example 1% of the data means a threshold of s/100).
- **One pass** (or none). It can give **false positives or false negatives**, so a verification pass may be needed.

(f) SON algorithm (Savasere, Omiecinski and Navathe) - two passes

- **Pass 1:** divide the data into **chunks** that fit in memory. Find the frequent itemsets **in each chunk** with the threshold $p \cdot s$ (where p is the fraction of the data in the chunk). Their **union** is the set of candidates.
- **Pass 2:** count all the candidates over the **whole dataset** and keep those with support ≥ s.
- **No false negatives**, because an itemset frequent overall must be frequent in at least one chunk. It **parallelises well with MapReduce**.

(g) Toivonen's algorithm

- Mine a **sample** with a **slightly lower threshold**. Also compute the **negative border** (itemsets that are not frequent, but whose immediate subsets all are).
- In a full pass, count the candidates **and** the negative border. If **no negative-border itemset is frequent**, the result is exact. Otherwise, **repeat with a new sample**.

4. Key Characteristics - Comparison

Algorithm	Passes	Main Idea	False Positives / Negatives
A-Priori	k + 1	Level-wise counting	Exact
PCY	2	Hash pairs into buckets + bitmap	Exact
Multistage	3 or more	Several sequential hash tables	Exact
Multihash	2	Several hash tables in one pass	Exact
Random sampling	1 (+1 to verify)	Mine a sample in memory	Both possible
SON	**2**	Chunk-wise mining + global count	**Exact**
Toivonen	1 to 2 (may repeat)	Sample + negative border	Exact if it succeeds

5. Advantages & Disadvantages

Advantages

- **Reduce disk I/O and memory use**, so mining terabyte-scale baskets becomes possible.
- SON and PCY are **exact** and **parallelisable** (MapReduce).

Disadvantages

- Hash-based methods need **careful sizing** of the hash tables.
- Sampling methods can **miss itemsets** or need **extra passes**.

6. Conclusion / Application

These algorithms are used for **market-basket analysis on retail data at Walmart scale, web click-stream mining and plagiarism detection** with MapReduce or Spark clusters.

Unit 5: Frameworks, Hadoop & R Programming

Q1. Short Definitions (2 Marks Each)

(a) What is Sharding in databases?

- **Sharding** is **horizontal partitioning** of a database. The **rows of a large table are split across multiple servers (shards)**, and each shard holds a subset of the data with the **same schema**.
- A **shard key** (for example user_id, or a region) decides which shard stores each row. Methods: **range-based, hash-based, directory-based**.
- *Benefit:* **horizontal scalability**, faster queries and higher availability. Used in MongoDB, Cassandra and MySQL clusters.

(b) What is a NoSQL database?

- A **NoSQL ("Not only SQL") database** is a **non-relational, schema-flexible, horizontally scalable** database designed for **big, distributed and unstructured** data. It usually follows **BASE** (Basically Available, Soft state, Eventually consistent) instead of strict ACID.
- **Types:**
  - **Key-value:** Redis, DynamoDB.
  - **Document:** MongoDB, CouchDB.
  - **Column-family:** HBase, Cassandra.
  - **Graph:** Neo4j.

(c) What are the basic data types in R?

Data type	Example	`class()` output
Numeric	`x <- 12.5`	"numeric"
Integer	`x <- 12L`	"integer"
Character	`x <- "AKTU"`	"character"
Logical	`x <- TRUE`	"logical"
Complex	`x <- 3 + 2i`	"complex"
Raw	`x <- charToRaw("A")`	"raw"

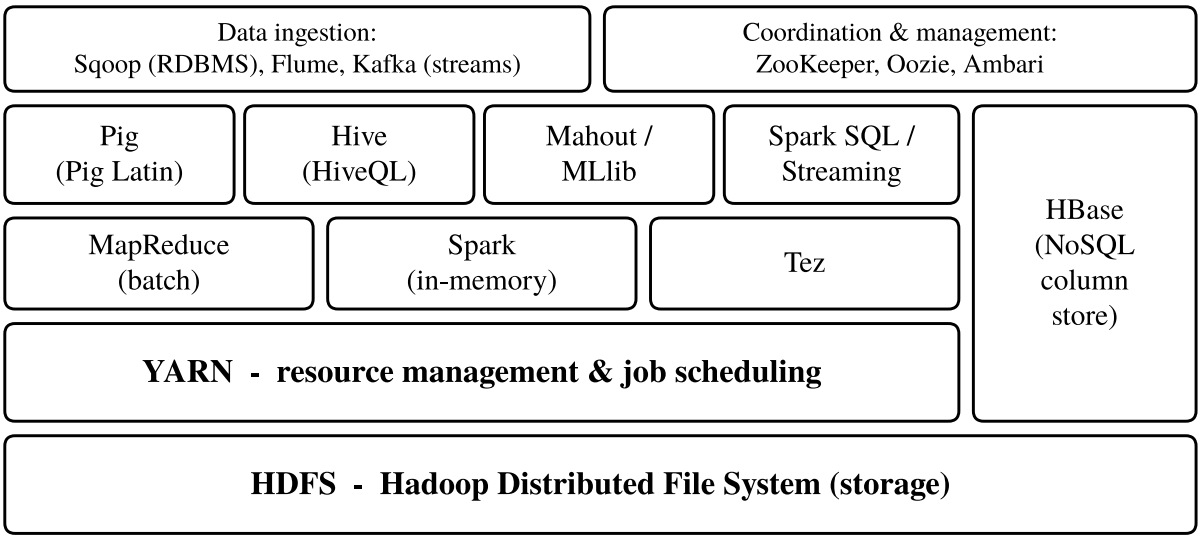
- *Data structures built on these types:* **vector, matrix, array, list, factor, data frame**.

Q2. Explain the Hadoop ecosystem and the architecture of the Hadoop Distributed File System (HDFS). (7 Marks)

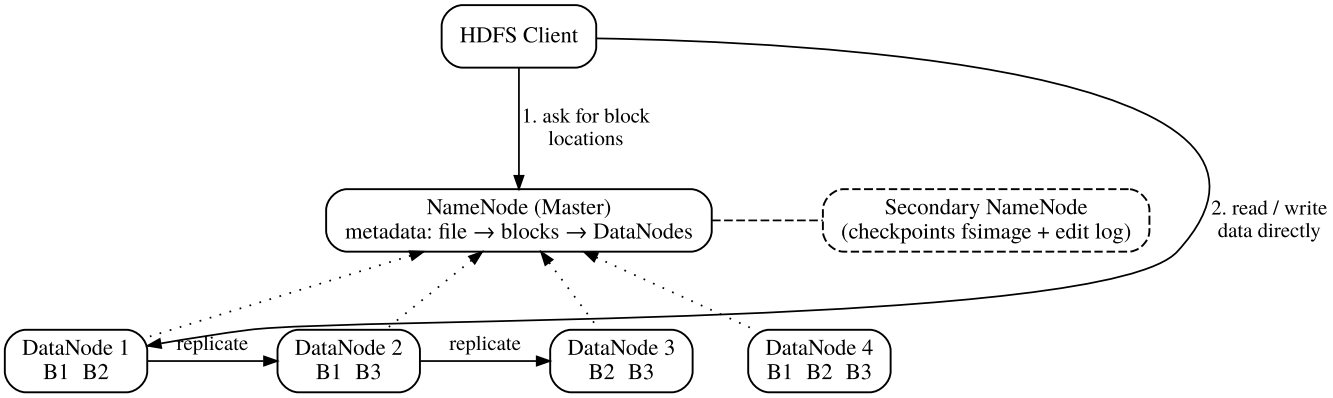
1. Introduction / Definition

Apache Hadoop is an **open-source framework** for the **distributed storage and processing of very large datasets** on clusters of **commodity hardware**. Its core is **HDFS** (storage), **YARN** (resource management) and **MapReduce** (processing), surrounded by an **ecosystem** of tools. **HDFS** is a **master-slave, block-based, fault-tolerant distributed file system**.

2. Block Diagrams



Hadoop ecosystem (layered view)



HDFS architecture (block size 128 MB, replication factor 3)

(Dotted arrows: heartbeats and block reports from each DataNode to the NameNode every 3 seconds.)

3. Core Explanation / Working

(a) Hadoop ecosystem components

Layer	Component	Role
Storage	**HDFS**	Distributed, replicated file storage
Storage	**HBase**	Column-oriented NoSQL database on HDFS (real-time reads and writes)
Resource management	**YARN**	Allocates CPU and memory; schedules jobs (ResourceManager + NodeManagers)
Processing	**MapReduce**	Batch parallel processing
Processing	**Spark**	In-memory processing, up to 100× faster
Data access	**Pig**	Data-flow scripting (Pig Latin)
Data access	**Hive**	SQL-like queries (HiveQL) on HDFS
Machine learning	**Mahout / Spark MLlib**	Scalable ML algorithms
Ingestion	**Sqoop**	Import and export between RDBMS and HDFS
Ingestion	**Flume / Kafka**	Streaming log and event ingestion
Coordination	**ZooKeeper**	Configuration, synchronisation, leader election
Workflow	**Oozie**	Job scheduling and workflows
Management	**Ambari**	Cluster provisioning and monitoring

(b) HDFS architecture

- **NameNode (master):**
  - Stores the **metadata**: the directory tree, the file-to-block mapping and the **block locations**.
  - Keeps the metadata in RAM, with persistent copies in the **fsimage** and the **edit log**.
  - Does **not** store the actual data.
- **DataNodes (slaves):**
  - Store the actual **data blocks** on their local disks.
  - Send **heartbeats** (every 3 seconds) and **block reports** to the NameNode.
  - Serve **read and write** requests from clients.
- **Secondary NameNode:**
  - Periodically **merges the edit log with the fsimage** (checkpointing), so the NameNode restarts faster.
  - It is **not a hot backup**. In Hadoop 2 and later, a **Standby NameNode** gives high availability.
- **Blocks:** files are split into large blocks (**128 MB by default**), which reduces metadata and seek overhead.
- **Replication:** each block is stored on **3 DataNodes by default**.
  - **Rack awareness:** one replica is on the local rack and two are on a different rack, so data survives a rack failure.

(c) Write operation

1. The client asks the NameNode to create the file.
2. The NameNode checks permissions and returns the **DataNodes for each block**.
3. The client writes the block to the first DataNode, which **pipelines** it to the second and third.
4. Acknowledgements flow back. The NameNode records the metadata.

(d) Read operation

1. The client asks the NameNode for the **block locations**.
2. It then reads each block **directly from the nearest DataNode**.
3. If a DataNode fails, the client reads from another replica.

(e) Fault tolerance

- If heartbeats from a DataNode stop, the NameNode marks it **dead** and **re-replicates** its blocks onto other nodes.

4. Key Characteristics - HDFS vs. Traditional File System

Basis	HDFS	Traditional FS (NTFS, ext4)
Storage	Distributed across a cluster	Single machine
Block size	**128 MB**	4 KB
Fault tolerance	**Replication** (3 copies)	RAID / backups
Access pattern	**Write once, read many**, streaming	Random read and write
Scalability	Thousands of nodes, petabytes	Limited by one machine
Hardware	Commodity	Any

5. Advantages & Disadvantages

Advantages

- **Scalable, cost-effective** storage for petabytes on cheap hardware.
- **Fault-tolerant** through replication. **Data locality** gives fast processing.
- A **rich ecosystem** for SQL, scripting, ML and streaming.

Disadvantages

- **Not suitable for many small files**, because the NameNode memory is a bottleneck.
- **High latency**. It does not suit real-time random access (use HBase for that).
- The NameNode can be a **single point of failure** without HA configuration.

6. Conclusion / Application

The Hadoop ecosystem is used by **Yahoo, Facebook, LinkedIn and banks** for log processing, data warehousing, recommendation systems and large-scale ETL.

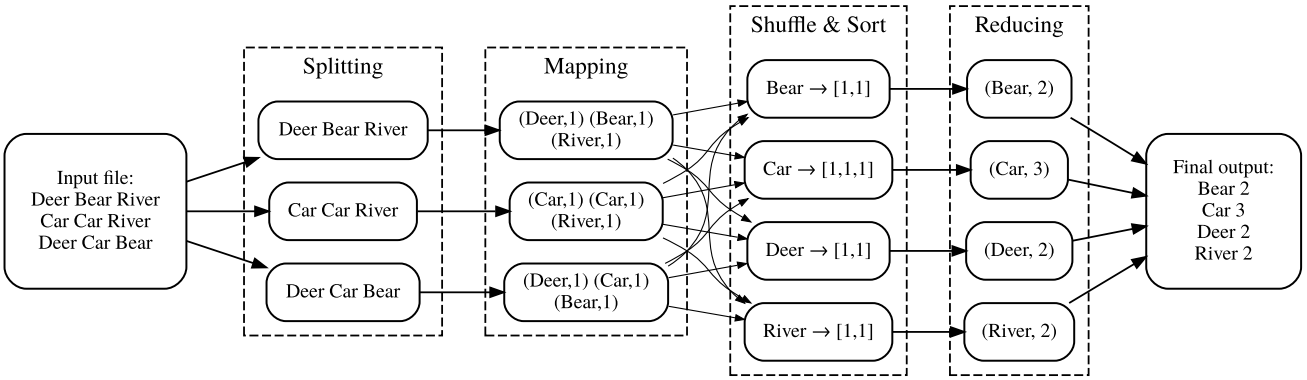
Q3. Detail the MapReduce framework with a block diagram (mapping and reducing phases). (7 Marks)

1. Introduction / Definition

MapReduce is a **programming model and processing framework** (Google, 2004; implemented in Hadoop) for **processing large datasets in parallel** across a cluster. A job has two user-defined functions: **Map**, which turns input records into **intermediate key-value pairs**, and **Reduce**, which **aggregates all the values for each key**. The

framework handles **splitting, shuffling, sorting, scheduling and fault tolerance**.

2. Block Diagram - Word Count Example



MapReduce data flow for Word Count

3. Core Explanation / Working

(a) Phases of a MapReduce job

1. **Input splitting:** the input in HDFS is divided into **input splits** (normally one per block, 128 MB). Each split goes to **one Map task**.
2. **Record reader:** converts each split into **(key, value)** records. For text input this is (byte offset, line).
3. **Map phase:**
   - The user's **map()** function runs on each record **in parallel** and outputs **intermediate (k, v) pairs**.
   - `map(k1, v1) → list(k2, v2)`. In word count, for each word w, emit (w, 1).
4. **Combiner (optional, a "mini-reducer"): aggregates locally** on the mapper node, for example (Car, 2) instead of two (Car, 1). This reduces network traffic.
5. **Partitioner:** decides **which reducer** gets each key. Default: `hash(key) mod R`.
6. **Shuffle and sort:**
   - Intermediate data is **transferred** from the mappers to the reducers.
   - It is **sorted by key**, and the values for the same key are **grouped**: (Car, [1, 1, 1]).
7. **Reduce phase:**
   - The user's **reduce()** function processes each key and its list of values.
   - `reduce(k2, list(v2)) → list(k3, v3)`. In word count: (w, sum of the list).
8. **Output:** the results are written to **HDFS** (one part-file per reducer).

(b) Pseudo-code for Word Count

```
map(String key, String line):
    for each word w in line:
        emit(w, 1)

reduce(String word, Iterator<int> counts):
    sum = 0
    for each c in counts:
        sum = sum + c
    emit(word, sum)
```

(c) Execution in Hadoop (YARN)

- The **ResourceManager** allocates containers.
- An **ApplicationMaster** coordinates the job's map and reduce tasks.
- **NodeManagers** run the tasks **close to the data** (data locality).
- **Failed tasks are re-run automatically** on other nodes.

4. Key Characteristics - Map Phase vs. Reduce Phase

Basis	Map Phase	Reduce Phase
Input	(k1, v1) records from the input split	(k2, list(v2)) grouped by key
Output	Intermediate (k2, v2) pairs	Final (k3, v3) pairs
Purpose	**Filter, transform, extract**	**Aggregate, summarise, combine**
Number of tasks	One per input split (many)	Set by the user (fewer)
Output stored	Local disk (temporary)	**HDFS** (permanent)
Runs	In parallel, with data locality	After shuffle and sort

5. Advantages & Disadvantages

Advantages

- **Massive parallelism** on commodity clusters, with **automatic fault tolerance**.
- **Simple programming model**: only map() and reduce() are needed.
- **Data locality** reduces network traffic. It scales to petabytes.

Disadvantages

- **High latency** because of disk writes between the phases. It does not suit real-time or iterative jobs such as ML (Spark is better).
- Complex jobs need **chains of many MapReduce jobs**, and the shuffle is expensive.

6. Conclusion / Application

MapReduce is used for **log analysis, web indexing (Google's original use), ETL, word counts, inverted indexes and large-scale sorting**.

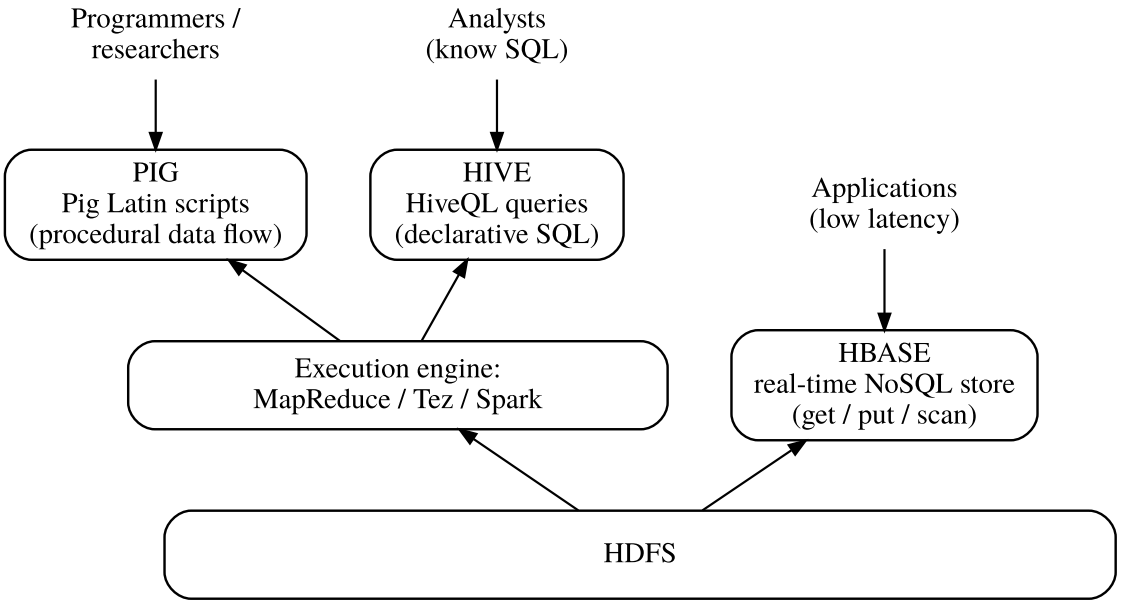
Q4. Differentiate between Pig, Hive and HBase. (7 Marks)

1. Introduction / Definition

Pig, Hive and HBase are Hadoop ecosystem tools with different purposes:

- **Apache Pig** is a **high-level data-flow scripting platform** (the Pig Latin language) for ETL.
- **Apache Hive** is a **data-warehouse system** that provides **SQL-like querying (HiveQL)** over data in HDFS.
- **Apache HBase** is a **distributed, column-oriented NoSQL database** on HDFS for **real-time random read/write** access.

2. Block Diagram



Where Pig, Hive and HBase sit in Hadoop

3. Core Explanation / Working

(a) Pig

- **Pig Latin** is a **procedural, step-by-step** data-flow language (LOAD, FILTER, GROUP, JOIN, FOREACH, STORE).
- Scripts are compiled into **MapReduce, Tez or Spark jobs**.
- Works on **structured and semi-structured** data, and is good for **ETL and data cleaning**.

```
logs   = LOAD '/data/logs' USING PigStorage(',') AS (user, url, time);
grp    = GROUP logs BY user;
counts = FOREACH grp GENERATE group, COUNT(logs);
STORE counts INTO '/out/usercounts';
```

(b) Hive

- **HiveQL** is a **declarative, SQL-like** language. Tables and a schema are defined over HDFS files (**schema-on-read**), and the metadata is kept in the **Metastore**.
- Queries become **batch jobs**, which suits **reporting and analytics** on large historical data.
- Supports **partitioning and bucketing**.

```
CREATE TABLE sales (id INT, region STRING, amount DOUBLE)
  ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';
SELECT region, SUM(amount) FROM sales GROUP BY region;
```

(c) HBase

- Modelled on **Google BigTable**. Data is stored in **tables → rows (row key) → column families → columns → versioned cells (timestamp)**.
- Gives **real-time random read/write**, millisecond access to single rows, and supports **billions of rows and millions of columns**.
- Architecture:

    - **HMaster** manages the cluster.
    - **RegionServers** serve the **regions** (sorted row-key ranges).
    - **ZooKeeper** handles coordination.
- Operations: `put`, `get`, `scan`, `delete`. There are **no joins and no SQL**, and transactions are limited to a single row.

4. Key Characteristics - Comparison Table

Basis	Pig	Hive	HBase
Type	Data-flow **scripting platform**	**Data warehouse** / SQL engine	**NoSQL column-family database**
Language	**Pig Latin** (procedural)	**HiveQL** (declarative, SQL-like)	Java API, HBase shell (get/put/scan)
Developed by	Yahoo	Facebook	Powerset (modelled on Google BigTable)
Processing	**Batch**	**Batch** (interactive with Tez/LLAP)	**Real-time** random access
Latency	High (minutes)	High (seconds to minutes)	**Low (milliseconds)**
Schema	Optional (defined in the script)	Schema-on-read (Metastore)	**Schema-less** columns (fixed column families)
Data model	Bags, tuples, fields	Tables, partitions, buckets	Row key → column family → column → version
Updates / deletes	No (works on files)	Limited (ACID tables)	**Yes**, row-level
Runs on	MapReduce / Tez / Spark	MapReduce / Tez / Spark	Directly on HDFS (RegionServers)
Best for	**ETL, data cleaning**, research prototyping	**Reporting, BI, ad-hoc analytics**	**Online apps**: messaging, time series, user profiles
Users	Programmers	SQL analysts	Application developers

5. Advantages & Disadvantages

- **Pig:** *advantages:* far less code than raw MapReduce, handles messy data. *Disadvantages:* a new language to learn, batch only.
- **Hive:** *advantages:* familiar SQL, a quick start for analysts. *Disadvantages:* high latency, not for OLTP or row-level updates.
- **HBase:** *advantages:* real-time access, massive scale, strong consistency. *Disadvantages:* no SQL or joins, row-key design is critical, complex to manage.

6. Conclusion / Application

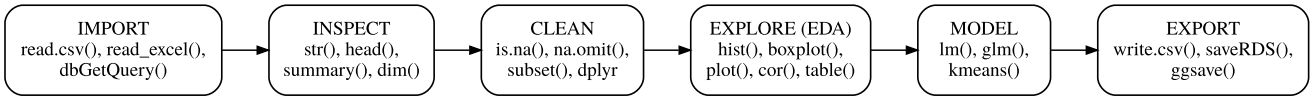
In a typical pipeline, **Pig cleans the raw logs, Hive runs the business reports**, and **HBase serves real-time lookups**. Facebook Messenger, for example, originally ran on HBase.

Q5. Explain data import/export and exploratory data analysis (EDA) using R. (7 Marks)

1. Introduction / Definition

R is an open-source **statistical programming language** widely used in data analytics. **Data import/export** means **reading data into R** from files, databases and the web, and **writing results out**. **Exploratory Data Analysis (EDA)** means **summarising and visualising data** to understand its structure, find patterns, outliers and relationships, and check assumptions before modelling.

2. Block Diagram - R Workflow



Data analysis workflow in R

3. Core Explanation / Working

(a) Importing data into R

```
# CSV and text files
df   <- read.csv("students.csv", header = TRUE, stringsAsFactors = FALSE)
df2  <- read.table("data.txt", sep = "\t", header = TRUE)

# Excel files (package readxl)
library(readxl)
xl   <- read_excel("marks.xlsx", sheet = 1)

# Databases (packages DBI + RMySQL / RSQLite)
library(DBI)
con  <- dbConnect(RMySQL::MySQL(), dbname = "college", user = "root", password = "****")
res  <- dbGetQuery(con, "SELECT * FROM results")

# JSON and web data
library(jsonlite)
js   <- fromJSON("https://api.example.com/data.json")

# Built-in datasets
data(iris)
```

(b) Exporting data from R

```
write.csv(df, "clean_students.csv", row.names = FALSE)
write.table(df, "out.txt", sep = "\t")
library(writexl);  write_xlsx(df, "report.xlsx")
saveRDS(df, "students.rds")        # R native format;  readRDS() to load
dbWriteTable(con, "clean_results", df)
```

(c) Exploratory Data Analysis in R (using the built-in `iris` dataset)

```
str(iris)          # structure: 150 obs. of 5 variables
head(iris, 5)      # first 5 rows
dim(iris)          # 150 5
summary(iris)      # min, 1st Qu., median, mean, 3rd Qu., max of each column
colSums(is.na(iris))   # missing values per column

mean(iris$Sepal.Length);  sd(iris$Sepal.Length);  var(iris$Sepal.Length)
table(iris$Species)                    # frequency of each category
cor(iris[, 1:4])                       # correlation matrix

hist(iris$Sepal.Length, main = "Histogram", xlab = "Sepal Length")   # distribution
boxplot(Sepal.Length ~ Species, data = iris)                         # outliers
plot(iris$Petal.Length, iris$Petal.Width)                            # scatter plot
pairs(iris[, 1:4])                                                   # scatter matrix

library(ggplot2)
ggplot(iris, aes(Petal.Length, Petal.Width, colour = Species)) + geom_point()
```

(d) EDA steps

1. **Understand the structure:** dimensions, data types, variable names.
2. **Univariate analysis:** summary statistics, histograms, boxplots.
3. **Bivariate and multivariate analysis:** scatter plots, correlation matrix, cross-tabulation.
4. **Detect missing values and outliers**, and decide how to treat them.
5. **Form hypotheses** for the modelling stage.

4. Key Characteristics - Common R Functions

Task	Function(s)
Import CSV / text	`read.csv()`, `read.table()`, `readr::read_csv()`
Import Excel / DB / JSON	`readxl::read_excel()`, `DBI::dbGetQuery()`, `jsonlite::fromJSON()`
Export	`write.csv()`, `write.table()`, `saveRDS()`, `writexl::write_xlsx()`
Structure	`str()`, `dim()`, `names()`, `head()`, `tail()`
Summary	`summary()`, `mean()`, `median()`, `sd()`, `quantile()`, `table()`
Missing values	`is.na()`, `na.omit()`, `complete.cases()`
Visualization	`hist()`, `boxplot()`, `plot()`, `barplot()`, `pairs()`, `ggplot2`
Relationships	`cor()`, `cov()`, `xtabs()`

5. Advantages & Disadvantages

Advantages

- **Free**, with **thousands of packages (CRAN)** for statistics and visualization.
- **Excellent graphics** (ggplot2), and EDA needs **very little code**.
- Connects to **many data sources**: files, databases, web APIs, Hadoop (RHadoop, sparklyr).

Disadvantages

- **In-memory**: very large data needs extra packages (data.table, sparklyr).
- **Steep learning curve**, and it is slower than compiled languages for some tasks.

6. Conclusion / Application

R is used for **statistical reporting in pharma (clinical trials), finance risk models, academic research and marketing analytics**.

Numerical Problems (PYQ Templates)

AKTU BCS-052 numericals come from Units 2, 3 and 4. Each answer below follows the same pattern: formula, substitution, answer.

N1. PCA: variance explained from eigenvalues (Unit 2)

Question: a dataset has eigenvalues 4.5, 2.8, 1.2, 0.4 and 0.1. (i) Calculate the percentage variance of each component. (ii) How many components are needed to retain 90% of the variance?

Formula

$$\text{Variance explained by } PC_k = \frac{\lambda_k}{\sum_i \lambda_i} \times 100\%$$

Step 1: Total variance

$$\sum \lambda_i = 4.5 + 2.8 + 1.2 + 0.4 + 0.1 = 9.0$$

Step 2: Percentage and cumulative variance

Component	Eigenvalue λ	% Variance = λ / 9.0 × 100	Cumulative %
PC1	4.5	$\frac{4.5}{9.0} \times 100 = 50.0\%$	50.0%
PC2	2.8	$\frac{2.8}{9.0} \times 100 = 31.1\%$	81.1%
PC3	1.2	$\frac{1.2}{9.0} \times 100 = 13.3\%$	**94.4%**
PC4	0.4	$\frac{0.4}{9.0} \times 100 = 4.4\%$	98.9%
PC5	0.1	$\frac{0.1}{9.0} \times 100 = 1.1\%$	100.0%

Answer: two components give only 81.1%, which is less than 90%. Three components give 94.4%, which is more than 90%. So **3 principal components are required** to retain 90% of the variance.

N2. PCA on 2-D patterns: (2,1), (3,5), (4,3), (5,6), (6,7), (7,8) (Unit 2)

Step 1: Mean of each feature

$$\bar{x} = \frac{2 + 3 + 4 + 5 + 6 + 7}{6} = 4.5, \qquad \bar{y} = \frac{1 + 5 + 3 + 6 + 7 + 8}{6} = 5$$

Step 2: Centre the data (subtract the mean)

Point	$x - \bar{x}$	$y - \bar{y}$	$(x - \bar{x})^2$	$(y - \bar{y})^2$	$(x - \bar{x})(y - \bar{y})$
(2, 1)	−2.5	−4	6.25	16	10.0
(3, 5)	−1.5	0	2.25	0	0.0
(4, 3)	−0.5	−2	0.25	4	1.0
(5, 6)	0.5	1	0.25	1	0.5
(6, 7)	1.5	2	2.25	4	3.0
(7, 8)	2.5	3	6.25	9	7.5
Sum			**17.5**	**34**	**22.0**

Step 3: Covariance matrix (dividing by n = 6)

$$\text{Cov} = \frac{1}{n}\begin{bmatrix} \sum (x-\bar{x})^2 & \sum (x-\bar{x})(y-\bar{y}) \\ \sum (x-\bar{x})(y-\bar{y}) & \sum (y-\bar{y})^2 \end{bmatrix} = \frac{1}{6}\begin{bmatrix} 17.5 & 22 \\ 22 & 34 \end{bmatrix} = \begin{bmatrix} 2.92 & 3.67 \\ 3.67 & 5.67 \end{bmatrix}$$

Step 4: Eigenvalues, from $|\text{Cov} - \lambda I| = 0$

$$\begin{vmatrix} 2.92 - \lambda & 3.67 \\ 3.67 & 5.67 - \lambda \end{vmatrix} = 0 \;\Rightarrow\; (2.92 - \lambda)(5.67 - \lambda) - (3.67)^2 = 0$$

$$\lambda^2 - 8.58\,\lambda + 3.08 = 0 \;\Rightarrow\; \lambda = \frac{8.58 \pm \sqrt{8.58^2 - 4(3.08)}}{2} = \frac{8.58 \pm 7.83}{2}$$

$$\lambda_1 = 8.21, \qquad \lambda_2 = 0.38$$

Step 5: Eigenvector for $\lambda_1 = 8.21$ (this is the first principal component)

$$(2.92 - 8.21)\,e_1 + 3.67\,e_2 = 0 \;\Rightarrow\; e_2 = \frac{5.29}{3.67}\,e_1 = 1.44\,e_1$$

Normalising: $PC_1 = \frac{1}{\sqrt{1 + 1.44^2}}\begin{bmatrix} 1 \\ 1.44 \end{bmatrix} = \begin{bmatrix} 0.57 \\ 0.82 \end{bmatrix}$, $\qquad PC_2 = \begin{bmatrix} -0.82 \\ 0.57 \end{bmatrix}$ (perpendicular)

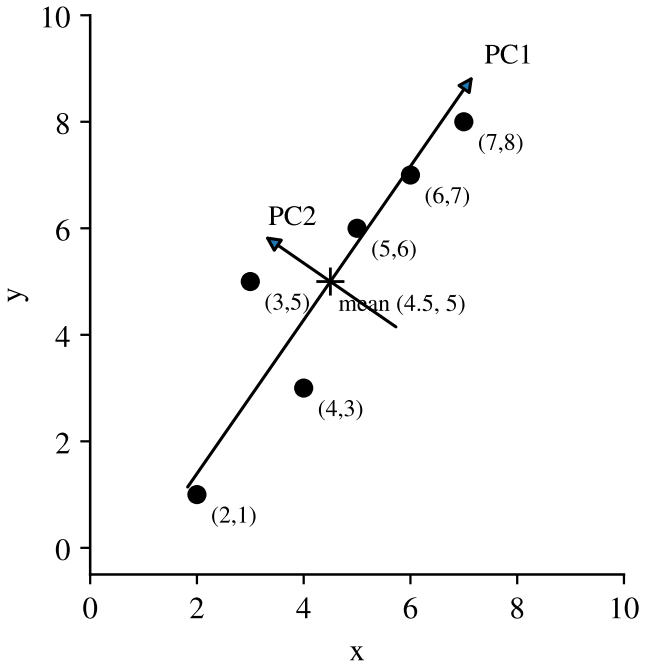
Step 6: Variance explained and projection

$$\frac{\lambda_1}{\lambda_1 + \lambda_2} = \frac{8.21}{8.59} = 95.6\%$$

- Projection of each point onto PC1: $z = 0.57\,(x - 4.5) + 0.82\,(y - 5)$

Point	(2,1)	(3,5)	(4,3)	(5,6)	(6,7)	(7,8)
z (1-D)	−4.71	−0.85	−1.93	1.11	2.50	3.89

Answer: PC1 = (0.57, 0.82), explaining 95.6% of the variance, and **PC2 = (−0.82, 0.57)**, explaining 4.4%. The 2-D data can be reduced to **1-D** (the z values above) with very little information loss.



Data points with the first principal component direction

(If your teacher divides by $n-1=5$, the covariance becomes [[3.5, 4.4], [4.4, 6.8]] and the eigenvalues become 9.85 and 0.45. The eigenvectors and the conclusion stay the same.)

N3. Linear regression: study hours (X) vs. scores (Y) (Unit 2)

Data: X = 2, 4, 6, 8, 10 and Y = 50, 55, 65, 70, 78. Find the regression equation and predict Y at X = 7.

Formulas

$$b_1 = \frac{\sum \left(X-\bar{X}\right)\left(Y-\bar{Y}\right)}{\sum \left(X-\bar{X}\right)^2}, \qquad b_0 = \bar{Y} - b_1\bar{X}, \qquad \widehat{Y} = b_0 + b_1 X$$

Step 1: Means

$$\bar{X} = \frac{30}{5} = 6, \qquad \bar{Y} = \frac{318}{5} = 63.6$$

Step 2: Table

X	**Y**	$X-\bar{X}$	$Y-\bar{Y}$	$\left(X-\bar{X}\right)\left(Y-\bar{Y}\right)$	$\left(X-\bar{X}\right)^2$
2	50	−4	−13.6	54.4	16
4	55	−2	−8.6	17.2	4
6	65	0	1.4	0	0
8	70	2	6.4	12.8	4
10	78	4	14.4	57.6	16
			Sum	**142**	**40**

Step 3: Coefficients

$$b_1 = \frac{142}{40} = 3.55, \qquad b_0 = 63.6 - 3.55 \times 6 = 42.3$$

(i) Regression equation: $\widehat{Y} = 42.3 + 3.55\,X$

(ii) Prediction at X = 7:

$$\widehat{Y} = 42.3 + 3.55 \times 7 = 42.3 + 24.85 = 67.15$$

- *Interpretation:* every extra hour of study adds about **3.55 marks**.

N4. Mean and standard deviation: {12, 15, 20, 22, 25, 30} (Unit 2)

Formulas

$$\bar{x} = \frac{\sum x_i}{n}, \qquad \sigma = \sqrt{\frac{\sum \left(x_i - \bar{x}\right)^2}{n}} \ \text{(population)}, \qquad s = \sqrt{\frac{\sum \left(x_i - \bar{x}\right)^2}{n-1}} \ \text{(sample)}$$

Step 1: Mean

$$\bar{x} = \frac{12 + 15 + 20 + 22 + 25 + 30}{6} = \frac{124}{6} = 20.67$$

Step 2: Squared deviations

x_i	**12**	**15**	**20**	**22**	**25**	**30**	**Sum**
$x_i - \bar{x}$	−8.67	−5.67	−0.67	1.33	4.33	9.33	
$(x_i - \bar{x})^2$	75.11	32.11	0.44	1.78	18.78	87.11	**215.33**

Step 3: Standard deviation

$$\sigma = \sqrt{\frac{215.33}{6}} = \sqrt{35.89} = 5.99, \qquad s = \sqrt{\frac{215.33}{5}} = \sqrt{43.07} = 6.56$$

Comment on the spread

$$\text{Coefficient of variation} = \frac{\sigma}{\bar{x}} \times 100 = \frac{5.99}{20.67} \times 100 = 29\%$$

- The SD of about 6 is moderate compared with the mean of about 20.7, and the values range from 12 to 30 (range = 18). The data shows **moderate spread** and is **fairly symmetric** (the median 21 is close to the mean 20.67), with no extreme outliers.

N5. Flajolet-Martin: S = {1, 3, 2, 1, 2, 3, 4, 3, 1, 2, 3, 1}, h(x) = (6x + 1) mod 5 (Unit 3)

Algorithm

1. For each element x, compute h(x) and write it in **binary**.
2. Let r(x) = the **number of trailing zeros** in that binary value.

3. R = the **maximum** r(x) over the whole stream.
4. **Estimated distinct count =** 2^R.

Step 1: Hash every element of the stream

Position	1	2	3	4	5	6	7	8	9	10	11	12
x	1	3	2	1	2	3	4	3	1	2	3	1
h(x)	2	4	3	2	3	4	0	4	2	3	4	2
Binary	010	100	011	010	011	100	000	100	010	011	100	010
r(x)	1	**2**	0	1	0	**2**	0*	**2**	1	0	**2**	1

- Working: $h(1) = 7 \mod 5 = 2, h(2) = 13 \mod 5 = 3, h(3) = 19 \mod 5 = 4,$ $h(4) = 25 \mod 5 = 0.$
- *Convention: a hash value of 0 is taken as r = 0 (it carries no trailing-one information).*

Step 2: Maximum trailing zeros: R = 2

Step 3: Estimate

$$\text{Distinct elements} \approx 2^R = 2^2 = \mathbf{4}$$

Answer: the FM estimate is **4 unique elements**. This matches the actual distinct set {1, 2, 3, 4}.

N6. Decaying window: 120,000 records per minute, 35% retained (Unit 3)

$$\text{Records retained per minute} = 120,000 \times \frac{35}{100} = \mathbf{42,000}$$

- So 78,000 records per minute (65%) are effectively faded out.

Justification for using a window

- A stream is **infinite**, so storing all 120,000 records every minute quickly exhausts memory.
- A **decaying window** gives each element a weight $(1-c)^{\text{age}}$. **Recent data matters most** and old data fades gradually, without a sudden cut-off.
- It needs only **one running sum per item**: $S \leftarrow S(1-c) + a_t$. This keeps memory and processing **constant**, while still tracking trends (for example the currently popular items).

N7. Sliding window: size 200, slide 50, after 1,000 elements (Unit 3)

Formula

$$\text{Number of windows} = \frac{N - W}{S} + 1$$

where N = elements processed, W = window size and S = slide step.

$$\text{Windows} = \frac{1000 - 200}{50} + 1 = \frac{800}{50} + 1 = 16 + 1 = \mathbf{17}$$

- The windows are [1-200], [51-250], [101-300], …, [801-1000].

Impact on latency

- A new result is produced **every 50 elements** (the slide), not every 200. This gives **lower output latency and fresher results**.
- Windows **overlap by 150 elements**, so each element is processed in up to 200/50 = **4 windows**. This means **more computation**, unless incremental updates are used.
- **Trade-off:** a smaller slide gives lower latency but higher CPU cost. A larger slide gives cheaper processing but staler results.

N8. Apriori: letter transactions with minimum support 60% (count ≥ 3) (Unit 4)

Transactions: T1 {A, B, C}, T2 {A, B}, T3 {A, C}, T4 {B, C}, T5 {A, B, C}

$$\text{min support count} = 60\% \times 5 = 3$$

Step 1: C1 → L1

Itemset	{A}	{B}	{C}
Count	4 (T1, T2, T3, T5)	4 (T1, T2, T4, T5)	4 (T1, T3, T4, T5)
Frequent?	Yes	Yes	Yes

Step 2: C2 → L2

Itemset	{A, B}	{A, C}	{B, C}
Count	3 (T1, T2, T5)	3 (T1, T3, T5)	3 (T1, T4, T5)
Frequent?	Yes	Yes	Yes

Step 3: C3 → L3

- Candidate {A, B, C}. Its subsets {A, B}, {A, C} and {B, C} are all frequent, so the Apriori property does **not** prune it.
- Count = 2 (T1, T5), which is less than 3, so it is **not frequent** and is removed at the support-counting step.

Answer: the frequent itemsets are **{A}, {B}, {C}, {A, B}, {A, C}, {B, C}**. There is no frequent 3-itemset.

Pruning shown: in this dataset the candidate {A, B, C} survives the **subset (Apriori) check** but fails the **support check**. If any 2-subset had been infrequent, it would have been pruned **without even counting it**.

N9. Apriori: grocery transactions, support = 50%, confidence = 60% (Unit 4)

TID	Items
T10	Beer, Nuts, Diaper
T20	Beer, Coffee, Diaper
T30	Beer, Diaper, Eggs
T40	Nuts, Eggs, Milk
T50	Nuts, Coffee, Diaper, Eggs, Milk

(This is the standard 5-transaction version from J. Han's textbook. If your paper has a 6th transaction, the method is exactly the same: just recount.)

$$\text{min support count} = 50\% \times 5 = 2.5 \;\Rightarrow\; 3$$

Step 1: C1 → L1

Item	Beer	Nuts	Diaper	Coffee	Eggs	Milk
Count	3	3	4	2	3	2
Frequent?	Yes	Yes	Yes	No	Yes	No

- L1 = {Beer}, {Nuts}, {Diaper}, {Eggs}

Step 2: C2 → L2

Pair	Beer, Nuts	Beer, Diaper	Beer, Eggs	Nuts, Diaper	Nuts, Eggs	Diaper, Eggs
Count	1	**3**	1	2	2	2
Frequent?	No	**Yes**	No	No	No	No

- L2 = {Beer, Diaper}. No 3-itemset can be frequent (C3 is empty), so the algorithm stops.

(i) Frequent itemsets: {Beer}, {Nuts}, {Diaper}, {Eggs}, **{Beer, Diaper}**

(ii) Strong association rules (confidence ≥ 60%)

$$\text{Support}(\text{Beer} \cup \text{Diaper}) = \frac{3}{5} = 60\%$$

$$\text{Confidence}(\text{Beer} \Rightarrow \text{Diaper}) = \frac{\text{count}(\text{Beer, Diaper})}{\text{count}(\text{Beer})} = \frac{3}{3} = 100\%$$

$$\text{Confidence}(\text{Diaper} \Rightarrow \text{Beer}) = \frac{3}{4} = 75\%$$

Rule	Support	Confidence	Strong (≥ 60%)?
Beer ⇒ Diaper	60%	100%	**Yes**
Diaper ⇒ Beer	60%	75%	**Yes**

N10. K-Means with Manhattan distance: 8 points, K = 3 (Unit 4)

Points: A1(2,10), A2(2,5), A3(8,4), A4(5,8), A5(7,5), A6(6,4), A7(1,2), A8(4,9) **Initial centres:** C1 = A1(2,10), C2 = A4(5,8), C3 = A7(1,2)

Formula

$$d(a,b) = |x_2 - x_1| + |y_2 - y_1|$$

Iteration 1

Point	d to C1 (2,10)	d to C2 (5,8)	d to C3 (1,2)	Cluster
A1 (2,10)	0	3 + 2 = 5	1 + 8 = 9	**C1**
A2 (2,5)	0 + 5 = 5	3 + 3 = 6	1 + 3 = **4**	**C3**
A3 (8,4)	6 + 6 = 12	3 + 4 = **7**	7 + 2 = 9	**C2**
A4 (5,8)	3 + 2 = 5	**0**	4 + 6 = 10	**C2**
A5 (7,5)	5 + 5 = 10	2 + 3 = **5**	6 + 3 = 9	**C2**
A6 (6,4)	4 + 6 = 10	1 + 4 = **5**	5 + 2 = 7	**C2**
A7 (1,2)	1 + 8 = 9	4 + 6 = 10	**0**	**C3**
A8 (4,9)	2 + 1 = 3	1 + 1 = **2**	3 + 7 = 10	**C2**

- Clusters: C1 = {A1}, C2 = {A3, A4, A5, A6, A8}, C3 = {A2, A7}

New centres after Iteration 1 (the answer usually asked in the PYQ)

$$C_1 = (2,\ 10)$$

$$C_2 = \left(\frac{8+5+7+6+4}{5},\ \frac{4+8+5+4+9}{5}\right) = (6,\ 6)$$

$$C_3 = \left(\frac{2+1}{2},\ \frac{5+2}{2}\right) = (1.5,\ 3.5)$$

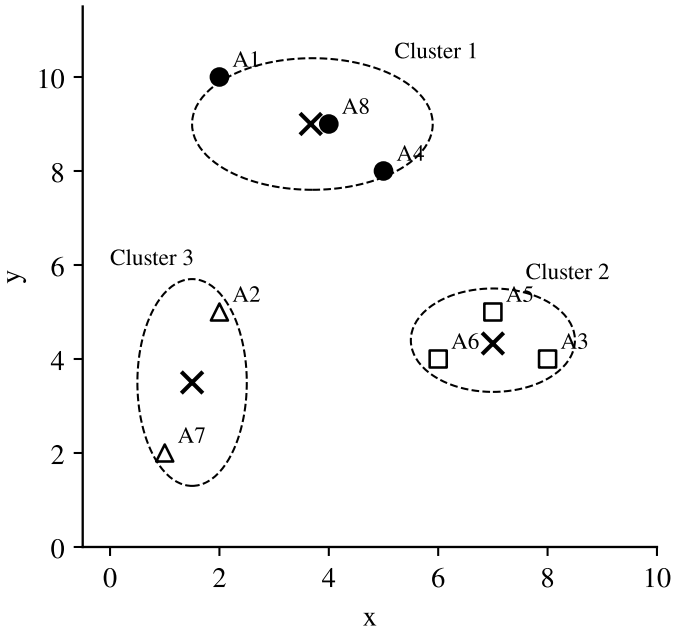
Iterations 2 to 4 (continue until nothing changes)

Iteration	Centres used	Clusters formed	New centres
2	(2,10), (6,6), (1.5,3.5)	C1 = {A1, A8}, C2 = {A3, A4, A5, A6}, C3 = {A2, A7}	(3, 9.5), (6.5, 5.25), (1.5, 3.5)
3	(3,9.5), (6.5,5.25), (1.5,3.5)	C1 = {A1, A4, A8}, C2 = {A3, A5, A6}, C3 = {A2, A7}	(3.67, 9), (7, 4.33), (1.5, 3.5)
4	(3.67,9), (7,4.33), (1.5,3.5)	**same as iteration 3**	no change, so the algorithm **converged**

Final answer

- **Cluster 1 = {A1, A4, A8}**, centre **(3.67, 9)**
- **Cluster 2 = {A3, A5, A6}**, centre **(7, 4.33)**

- **Cluster 3 = {A2, A7}**, centre (**1.5, 3.5**)



Final three clusters (x = final centre)
